

Dual Bachelor in Data Science and Engineering and
Telecommunication Technologies Engineering
2025-2026

Bachelor Thesis

Local Volatility Surface Calibration and Barrier Option Pricing via Physics-Informed Deep Learning

Álvaro Martín Ruiz

Emilio Parrado Hernández

Madrid, July 2026



This work is licensed under Creative Commons **Attribution – Non Commercial – Non Derivatives**

ABSTRACT

This thesis investigates the application of Physics-Informed Neural Networks (PINNs) for addressing the inverse problem of calibrating local volatility models for financial option pricing, that is, reconstructing the volatility surface from observable market prices. The thesis also shows the same architecture can be employed in the forward problem of pricing exotic derivatives once calibration is completed.

The methodology consists of four major phases. The first phase involves testing a PINN on pricing options under constant volatility and validating it by checking against the closed-form of Black–Scholes. In the second phase, the architecture is transformed to a dual-output network that simultaneously reconstructs both the local volatility surface and option prices on synthetic data, in which the true surface is known and accuracy of the reconstructions can be measured. The third phase of the methodology uses data from the real market to reconstruct a smooth, arbitrage-free local volatility surface using out-of-the-money (OTM) SPY options from 20 September 2021 and benchmarking the resulting surface against two classical alternatives: raw finite-difference Dupire inversion and SVI-smoothed Dupire inversion. Finally, in the fourth phase of the methodology, the calibrated local volatility surface is used as the known coefficient to price knock-out barrier options using three separate engines (Crank–Nicolson finite difference, PINN, and Monte Carlo) and cross-validate prices and compare option sensitivities (Greeks) across methods.

The PINN-calibrated surface achieves a price reconstruction $R^2 = 0.9991$ on the full market dataset and is the only method tested to produce a fully valid local volatility surface, with zero negative local variance across its domain, compared to 8.5% invalid cells for raw finite-difference Dupire inversion and 65.7% for SVI-smoothed Dupire inversion.

In addition, the barrier option prices produced by the PINN engine reproduce the finite-difference and Monte Carlo benchmarks for down-and-out calls, agreeing with both to within 2.2%, whereas up-and-out calls are identified as a failure mode of the method: the network does not converge reliably in that regime and the resulting prices deviate substantially from the benchmarks. Lastly, the Greek letters computed via automatic differentiation through the PINN produced smooth and logical sensitivities across the entire domain, in contrast to the finite-difference bump-and-reset Greeks, which exhibit oscillations near the barrier boundary.

In summary, PINNs allow to incorporate prior knowledge based upon the Black–Scholes Partial Differential Equation, while also satisfying all the constraints placed upon them, which makes it a robust method for the calibration of volatility and pricing of exotic derivatives.

Keywords: Physics-Informed Neural Networks; local volatility; option pricing; volatility surface calibration; Black–Scholes; Dupire equation; barrier options; finite-difference methods; Monte Carlo simulation; automatic differentiation; Greeks.

ACKNOWLEDGMENTS

To the two people who have had the greatest influence on both my academic and personal life, my mom and dad: everything I am is a result of your support and the values you instilled in me. Thank you.

To my brother, Adrian, for always answering my calls, no matter how unusual the topic I want to discuss may be. Thank you for always encouraging me to learn and grow.

To Tía Moni, for being my second mother over the last five years and for always cheering me up during the difficult times and hard work. I honestly do not know how you never kicked me out of your home. Thank you.

To my grandparents, uncles, cousin, and the rest of my family: I truly won the jackpot with you.

To my friends, both those who have been by my side for as long as I can remember and those who have joined me more recently, thank you.

To Irene, I still do not know why you choose to spend time with me, but I am never going to complain about it. Thank you.

To my teachers, for shaping the student I have become. Especially Antonio and Emilio, thank you.

CONTENTS

LIST OF ABBREVIATIONS	xii
1. INTRODUCTION.	1
1.1. Motivation and context	1
1.2. Problem statement	2
1.3. Research objectives and hypotheses	2
1.4. Scope and limitations	3
1.5. Structure of the document	4
2. BACKGROUND	5
2.1. Options and Derivatives	5
2.1.1. Vanilla European Options	5
2.1.2. Path-Dependent and Barrier Options	7
2.1.3. The Greeks	8
2.2. Pricing Theory	9
2.2.1. The Black–Scholes Model and PDE	9
2.2.2. Limitations of Constant Volatility: The Empirical Smile.	10
2.3. Local Volatility	11
2.3.1. Dupire’s Formula and the Inverse Problem	11
2.3.2. Why the Inverse Problem is Ill-Posed	11
2.3.3. Classical Solutions to the Ill-posed Problem.	12
2.4. Numerical Methods for Option Pricing	13
2.4.1. Finite Differences and Crank–Nicolson	13
2.4.2. Monte Carlo Simulation	14
2.4.3. The BGK Continuity Correction for Barriers	14
2.5. Neural Networks and Physics-Informed Learning	15
2.5.1. Neural Networks and Automatic Differentiation	15
2.5.2. Physics-Informed Neural Networks	16
2.5.3. Hard vs Soft Boundary Enforcement.	16

3. RELATED WORKS.	18
3.1. Classical Volatility Surface Calibration	18
3.2. Neural Network Approaches to Option Pricing	19
3.3. Physics-Informed Methods in Quantitative Finance	19
3.4. Barrier Option Pricing under Local Volatility	20
3.5. Research Gap and Positioning of This Thesis	20
4. METHODOLOGY	22
4.1. Overall pipeline architecture	22
4.2. Phase 1: Synthetic vanilla pricing (forward problem).	22
4.2.1. Network architecture.	23
4.2.2. Loss function design	23
4.2.3. Training procedure	24
4.3. Phase 2: Synthetic local volatility calibration (inverse problem)	24
4.3.1. The dual-output PINN	24
4.3.2. Synthetic local volatility surface	25
4.3.3. Loss function and architectural modifications	25
4.3.4. Training procedure	27
4.4. Phase 3: Real-data calibration on SPY	27
4.4.1. Data source and preprocessing	27
4.4.2. Vega-weighted data loss	27
4.4.3. Full loss function and training	28
4.4.4. Benchmark methods: raw FDM Dupire and SVI-smoothed Dupire	28
4.5. Phase 4: Barrier option pricing	29
4.5.1. Problem formulation and coordinate bridge	29
4.5.2. Crank–Nicolson FDM and Monte Carlo benchmarks.	30
4.5.3. Barrier PINN with squashed boundary ansatz	30
5. RESULTS	31
5.1. Phase 1: Synthetic vanilla pricing	31
5.2. Phase 2: Synthetic LV calibration	32
5.3. Phase 3: Real-data calibration on SPY	33
5.3.1. Calibration accuracy	33

5.3.2. Recovered local volatility surface.	34
5.3.3. Three-way comparison: raw FDM vs SVI-smoothed vs PINN	34
5.4. Phase 4: Barrier option pricing	36
5.4.1. Four-way pricing comparison	36
5.4.2. LV correction over Black-Scholes	38
5.4.3. Greeks smoothness and pricing speed	38
6. CONCLUSIONS AND FUTURE WORK	40
6.1. Summary of contributions	40
6.2. Overall Assessment	41
6.3. Future work	42
7. REGULATORY AND SOCIO-ECONOMIC ENVIRONMENT	44
7.1. Regulatory Framework	44
7.1.1. Financial regulation and model governance	44
7.1.2. EU Artificial Intelligence Act	45
7.1.3. Technical standards and open-source licensing	45
7.1.4. Intellectual property	45
7.2. Socio-economic Environment.	46
7.2.1. Project budget	46
7.2.2. Socio-economic impact	46
BIBLIOGRAPHY.	48
A. MATHEMATICAL DERIVATIONS	51
A.0.1. Derivation of the Black-Scholes PDE	51
A.0.2. Analytical Closed-Form Formulas for Barrier Options	52
A.0.3. The Broadie-Glasserman-Kou (BGK) Continuity Correction	53
B. IMPLEMENTATION DETAILS	55
C. CODE REPOSITORY STRUCTURE.	60

LIST OF FIGURES

2.1	Sensitivity analysis of the SVI parameter space. Each plot shows the impact of a single free parameter (a, b, ρ, m, ξ) on the implied variance smile $w(k)$	12
2.2	Computational graph for $y = \tanh(w \cdot x)$. The forward pass (left) calculates the intermediate variables from inputs to the final output. The backward pass (right) applies the chain rule from right to left.	15
2.3	The exponential squashed mask ansatz $g(m)$ used to hard-enforce the down-and-out barrier boundary condition.	17
4.1	Architecture of the coupled PINN model.	24
4.2	Synthetic local volatility surface used to generate the training data.	25
5.1	Phase 1 error heatmap	31
5.2	Phase 1 price slices	32
5.3	Phase 2 volatility surface comparison	32
5.4	Phase 2 smile cross-sections	33
5.5	Phase 3 price fit	33
5.6	Phase 3 recovered local volatility surface	34
5.7	Phase 3 three-way surface comparison	35
5.8	Phase 3 smile cross-sections at four maturities	35
5.9	Phase 4 barrier sensitivity curves	38
5.10	Phase 4 Greeks comparison	39
A.1	Visualization of the Reflection Principle for a DOC. A path starting at S_0 that crosses the barrier B has a symmetric path starting at B^2/S_0 (black) that also crosses B . The two distances d are equal: $d = \ln(S_0/B) = \ln(B/\frac{B^2}{S_0})$	53

LIST OF TABLES

5.1	PHASE 1 ERROR METRICS VS. BLACK–SCHOLES CLOSED-FORM SOLUTION	31
5.2	PHASE 3: SURFACE VALIDITY COMPARISON ACROSS METHODS	34
5.3	PHASE 4: FOUR-WAY BARRIER OPTION PRICE COMPARISON ($S_0 = K = \$434$, $T = 0.5$ YR)	36
7.1	PROJECT BUDGET	46
B.1	HARDWARE AND SOFTWARE ENVIRONMENT	55
B.2	NEURAL NETWORK ARCHITECTURES BY PHASE	55
B.3	PHASE 1 – BLACK–SCHOLES PARAMETERS AND TRAINING CONFIGURATION	56
B.4	PHASE 2 – SYNTHETIC INVERSE CALIBRATION CONFIGURATION	57
B.5	PHASE 3 – SPY MARKET CALIBRATION CONFIGURATION	58
B.6	PHASE 4 – BARRIER OPTION PINN AND BENCHMARK ENGINES	59
B.7	TOTAL TRAINING TIMES ON THE REFERENCE HARDWARE (INTEL CORE i3-1115G4, CPU ONLY)	59

LIST OF ABBREVIATIONS

AD	Automatic Differentiation.	ATM	At-the-Money.
BC	Boundary Condition.	BGK	Broadie–Glasserman–Kou continuity correction.
BS	Black–Scholes model/formula.	CN	Crank–Nicolson method.
DOC	Down-and-Out Call.	DTE	Days to Expiry.
FDM	Finite Difference Method.	FRTB	Fundamental Review of the Trading Book.
GBM	Geometric Brownian Motion.	IC	Initial Condition.
ITM	In-the-Money.	IV	Implied Volatility.
L-BFGS	Limited-memory Broyden–Fletcher–Goldfarb–Shanno optimizer.	LV	Local Volatility.
MAE	Mean Absolute Error.	MC	Monte Carlo.
MiFID	Markets in Financial Instruments Directive.	MLP	Multi-Layer Perceptron.
MSE	Mean Squared Error.	NN	Neural Network.
OTC	Over-the-Counter.	OTM	Out-of-the-Money.
PDE	Partial Differential Equation.	PINN	Physics-Informed Neural Network.
RMSE	Root Mean Squared Error.	SABR	Stochastic Alpha–Beta–Rho model.
SDE	Stochastic Differential Equation.	SPY	SPDR S&P 500 ETF.
SVI	Stochastic Volatility Inspired parametrization.	UOC	Up-and-Out Call.

1. INTRODUCTION

1.1. Motivation and context

Option pricing is one of the most challenging tasks in mathematical finance. It requires solving partial differential equations (PDEs) that are often computationally expensive, subject to complex boundary conditions, and, in some cases, lack closed-form solutions altogether.

One of the most fundamental difficulties lies in recovering the **volatility surface**: a latent function that governs the dynamics of option prices but cannot be observed directly. Instead, it must be inferred from market-quoted prices, a process known as **calibration**. Recovering this surface is an **ill-posed inverse problem**: the solution is not unique, depends discontinuously on the input data, and must be regularised to obtain a stable, financially meaningful result. Small perturbations in observed prices can produce large swings in the recovered volatility, making classical numerical approaches very fragile.

The financial stakes are substantial. The global equity-options market alone 'moves' hundreds of billions of dollars each day, and even small systematic errors in volatility calibration translate into mispricing, hedging losses, and regulatory capital implications that may lead to millions of dollars in sanctions.

Most institutions today rely on the Black-Scholes framework, augmented with smile-fitting extensions such as **local volatility** [1] or **stochastic volatility** models (Heston [2], SABR [3]), despite the well-documented limitations of these approaches.

The recent emergence of **Physics-Informed Neural Networks (PINNs)** [4] offers an alternative methodology. By embedding the governing PDE directly into the loss function of a neural network, PINNs can solve forward and inverse problems within a single framework, handle noisy or sparse data through implicit regularisation, and produce smooth, differentiable solutions that are especially convenient for downstream tasks such as the computation of 'the Greeks'. Classical numerical methods (finite differences, finite elements, Monte Carlo) solve the forward pricing problem efficiently, but cannot handle the inverse calibration problem, where noise in the input prices is amplified by the numerical differentiation that the Dupire formula requires. PINNs circumvent this by learning the volatility surface that satisfies multiple conditions at the same time: it solves for a smooth function whose derivatives are jointly consistent with the PDE, the boundary conditions, and the observed data.

This thesis investigates whether a PINN-based pipeline can match or surpass the accuracy of classical methods on the computation of the option price, the Greeks and the calibration process.

1.2. Problem statement

This thesis addresses two interconnected problems that arise in equity-derivatives pricing:

1. **The calibration problem (inverse).** Given a set of market-quoted option prices on a single equity underlying (SPY), recover a local volatility surface $\sigma(m, \tau)$ that is smooth, free of static arbitrage, and consistent with the observed prices. The surface must be well-defined at every point in its domain, including data-sparse regions.
2. **The downstream pricing problem (forward).** Using the calibrated volatility surface as a known coefficient (we 'freeze' the volatility surface), price path-dependent exotic contracts (we tested knock-out barrier options) and compute their Greeks (Delta, Gamma). The challenge is not only to produce accurate prices (which classical finite-difference and Monte Carlo methods can already deliver) but to produce *smooth, analytically differentiable* sensitivities.

The two problems are solved one after the other: the output of the calibration phase (a volatility surface) is the input to the pricing phase.

1.3. Research objectives and hypotheses

The thesis pursues four objectives:

- O1. Validate the PINN framework on synthetic data (forward problem).** Train a PINN to price vanilla European call options under known constant volatility, and verify convergence against the Black-Scholes closed-form solution. This is just a baseline to check that the architecture works correctly.
- O2. Validate the PINN framework on synthetic data (inverse problem).** Extend the architecture with a dual-output network that simultaneously recovers a local volatility surface and fits synthetically-generated option prices, where the true surface is known and recovery accuracy can be measured directly.
- O3. Calibrate a local volatility surface from real SPY market data.** Apply that methodology to a real scenario. To make it easier, the chosen data comes from liquid out-of-the-money SPY options on a single snapshot date (20 September 2021), and compare the resulting surface against two classical alternatives: raw finite-difference Dupire inversion and SVI-smoothed Dupire inversion.
- O4. Price barrier options under the calibrated surface.** Implement three independent pricing engines: Crank-Nicolson finite differences (FDM), Monte Carlo with the Broadie-Glasserman-Kou continuity correction (MC), and a barrier-specific PINN, and cross-validate prices and Greeks across all three methods.

These objectives support two testable hypotheses:

- H1 (Price accuracy).** Barrier option prices computed from the PINN-calibrated local volatility surface by the classical FDM and MC engines agree with one another, confirming that the neural-network calibration introduces no systematic bias into the surface itself.
- H2 (Greeks quality).** The PINN framework produces smooth option sensitivities, especially Delta and Gamma, whereas finite-difference methods can produce Greeks that oscillate near the barrier boundary.

1.4. Scope and limitations

The following boundaries define the scope of this thesis:

- **Single underlying.** All experiments are conducted on SPY, the most liquid equity option market. The methodology is general, but has not been tested on other underlyings, asset classes, or markets.
- **Single snapshot date.** Calibration is performed on one end-of-day quote from 20 September 2021. Time-series analysis and recalibration dynamics are outside the scope of this work.
- **European exercise only.** SPY options are technically American-style, but the analysis filters for out-of-the-money contracts where the early-exercise premium is negligible. No American or Bermudan pricing engine is implemented.
- **Local volatility model.** The framework recovers a deterministic volatility surface $\sigma(K, T)$. Stochastic-volatility or local-stochastic-volatility extensions, which capture vol-of-vol effects and forward-smile dynamics, are not addressed.
- **Barrier options only.** Phase 4 prices knock-out barrier options (down-and-out and up-and-out calls). Other exotics are outside scope.
- **Constant interest rate and dividend yield.** Both r and q are treated as scalars interpolated from the Treasury curve and the trailing S&P 500 dividend yield, respectively. Full term-structure modelling is not implemented.
- **Single-barrier, one direction at a time.** Double-barrier (simultaneous up-and-down knock-out) and knock-in variants are not implemented.

1.5. Structure of the document

The remainder of this thesis is organised as follows.

Chapter 2 provides the theoretical background required to follow the methodology. It focuses on explaining financial concepts that may be unfamiliar for an engineer-related reader (vanilla and barrier options, the Greeks), the pricing theory (risk-neutral valuation, the Black-Scholes model and its limitations, local volatility and Dupire’s equation), the numerical methods employed (Crank-Nicolson finite differences, Monte Carlo simulation with variance reduction and the Broadie-Glasserman-Kou correction), and the machine-learning foundations (neural networks, automatic differentiation, and Physics-Informed Neural Networks).

Chapter 3 reviews the existing literature on volatility surface calibration and PINN-based methods, from classical Dupire-based approaches to deep calibration and physics-informed learning, and identifies the research gaps this thesis addresses.

Chapter 4 describes the four-phase experimental pipeline in detail. Phase 1 validates the PINN on the forward pricing problem under constant volatility. Phase 2 extends the architecture to the inverse calibration problem using a custom parametric local volatility surface as ground truth. Phase 3 applies the methodology to real SPY market data, including the full preprocessing pipeline, vega-weighted loss design, and two-stage training protocol. Phase 4 prices barrier options under the calibrated surface using three independent engines and a barrier-specific PINN.

Chapter 5 presents the experimental results: calibration accuracy, surface validity comparisons, barrier price cross-validation, sensitivity curves, Greeks quality, and pricing speed benchmarks.

Chapter 6 summarises the contributions of this thesis, maps the experimental outcomes to the stated objectives and hypotheses, and outlines directions for future work, including parametric barrier PINNs, extended PINNs with domain partitions, American-exercise extensions, and stochastic-local-volatility hybrids.

Chapter 7 discusses the regulatory and socio-economic environment surrounding a production deployment of the methodology. The legal framework in which this project is embedded is extremely important, as banks and investment funds face strict regulations, where committing infractions may result in sanctions amounting to millions of dollars in fines.

Supporting material, including mathematical derivations, full hyperparameter tables, additional figures, and the repository structure, is provided in the appendices.

2. BACKGROUND

This chapter presents the theoretical foundations required to properly understand everything that will be discussed in this thesis. It is expected that the reader has a graduate-engineering level, so basic mathematical or ML concepts are expected to be familiar for the reader. A special emphasis is given to the financial concepts, as it may be an unfamiliar research field.

2.1. Options and Derivatives

A **financial derivative** is a contract whose value depends on the future evolution of one (or more) underlying assets, usually a stock, a commodity or an interest rate. Derivatives are used for hedging (reducing exposure to an unwanted risk), speculation (betting on the future direction of the asset), and arbitrage (strategies that exploit market discrepancies with no risk). The most basic kind of derivative (along with the future contract) is the option.

An **option** is a contract that gives its holder the *right*, but not the *obligation*, to buy or sell an underlying asset at a fixed and pre-agreed price on a specified date called maturity (in some cases, also at dates before the maturity). The key parameters are:

- **Underlying asset S** : the price of the asset on which the option depends.
- **Strike price K** : the agreed price at which the holder can buy or sell the underlying.
- **Expiry or maturity T** : the date on which the contract matures.
- **Premium**: the price paid upfront by the buyer to acquire the option.

The two basic types of options are the ones that give the right to *buy*, **call options**, and those who give the right to *sell*, **put options**. Depending on the price of the underlying, an option is said to be **in-the-money (ITM)** if immediate exercise would be profitable, **at-the-money (ATM)** if the spot price is close to the strike, and **out-of-the-money (OTM)** if immediate exercise would not be profitable.

2.1.1. Vanilla European Options

A **vanilla European option** is the simplest option contract. It can only be exercised at expiry T , and its payoff depends just on the value of the underlying asset at that time. The payoffs of a European call and put are:

$$C_T = \max(S_T - K, 0), \quad (2.1)$$

$$P_T = \max(K - S_T, 0). \quad (2.2)$$

Essentially, a call pays the difference between the spot price and the strike if the underlying price is above the strike; otherwise it expires worthless. The put pays the difference in the opposite direction (the opposite of a call).

Moneyiness. It is convenient to express the strike relative to the current spot price. The **log-moneyness** is defined as:

$$m = \ln\left(\frac{S_0}{K}\right), \quad (2.3)$$

where S_0 is the current spot price. Positive m corresponds to ITM calls (and OTM puts); negative m corresponds to OTM calls (and ITM puts). This is used in the project as it ensures the information provided is scale-invariant.

Put–call parity. For European options on an asset paying a continuous dividend yield q (an approximation where discrete dividend payments are treated as a continuous, constant fraction of the asset price over time), the prices of a call and a put with the same strike and expiry satisfy:

$$C - P = S_0 e^{-qT} - K e^{-rT}, \quad (2.4)$$

where r is the continuously compounded risk-free rate. Put–call parity is used in this thesis to convert observed put prices into call-equivalent prices.

American options. American options may be exercised at any time up to and including expiry (contrary to European options, that could only be exercised at expiry). This early-exercise feature introduces an additional dimension of complexity: the option value is the maximum of the immediate exercise value and the continuation value, which creates a complex free-boundary problem. SPY options are American-style, but this thesis restricts attention to OTM contracts where the early-exercise premium is negligible and can be approximately treated as European.

Bermudan options. These options are an intermediate class between European and American styles. Bermudan options allow early exercise only on a set of specified, discrete dates before expiry. They therefore allow greater flexibility than European contracts but less than American ones.

2.1.2. Path-Dependent and Barrier Options

A **path-dependent** option is one whose payoff depends also on the trajectory (path) followed by the underlying over the life of the contract. This makes path-dependent options much harder to price, as the value is no longer a function of (S_T, T) alone.

Barrier options are the most common path-dependent options. They are European options with an additional feature: the contract is activated (**knock-in**) or extinguished (**knock-out**) if the underlying crosses a pre-specified **barrier level** B at any point during the life of the option. This thesis focuses on knock-out calls, which are divided into:

- **Down-and-out call** ($B < S_0$): the option is knocked out (becomes worthless) if the spot price falls below the barrier at any time before expiry. Its payoff is:

$$C_T^{\text{DO}} = \max(S_T - K, 0) \cdot \mathbf{1}_{\left\{ \min_{t \in [0, T]} S_t > B \right\}}. \quad (2.5)$$

- **Up-and-out call** ($B > S_0$): the call is knocked out if the spot price rises above the barrier at any time before expiry. Its payoff is:

$$C_T^{\text{UO}} = \max(S_T - K, 0) \cdot \mathbf{1}_{\left\{ \max_{t \in [0, T]} S_t < B \right\}}. \quad (2.6)$$

It is then logical that barriers have a lower premium than European options, as they have the same characteristics but the knock-out barrier introduces a higher probability of becoming worthless.

Pricing PDE for barrier options. For a knock-out call, the PDE is the same Dupire or BS equation as for a vanilla option, but the spatial domain is truncated at the barrier and the condition $V(B, t) = 0$ is imposed there for all $t < T$, where $V(B, t)$ represents the value of the option when the underlying asset reaches the barrier level B at time t .

$$\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + (r - q)S \frac{\partial V}{\partial S} - rV = 0, \quad S \in \Omega_B, \quad (2.7)$$

$$V(S, T) = \max(S - K, 0), \quad (2.8)$$

$$V(B, t) = 0, \quad t < T, \quad (2.9)$$

where Ω_B is the spatial domain truncated at the barrier (e.g. $\Omega_B = (B, \infty)$ for a DOC).

Closed-form solution. Similar to vanilla options, barrier options admit a closed-form solution under constant volatility, derived by Rubinstein and Reiner [5]. This solution uses the reflection principle of Brownian motion (for a Brownian motion path that hits a barrier, there is an equally probable "mirrored" path on the other side of the barrier, see Annex A.0.2 for the explanation). For a down-and-out call with $K \geq B$, the price is:

$$C^{\text{DO}} = A - C, \quad (2.10)$$

where

- A is the Black-Scholes value of the call option. It represents the value if the barrier didn't exist.
- C is the "reflection" term. It represents the value subtracted from the option because of the probability of hitting the barrier and becoming worthless.

Therefore, the Down-and-Out price is the vanilla price minus the knock-out discount. The full expression can be found in the annex, it is used in this thesis as a preliminary benchmark in phase 4 to validate both the Crank–Nicolson FDM and MC implementations under constant volatility before testing on the calibrated surface.

2.1.3. The Greeks

The **Greeks** are partial derivatives of the option price w.r.t. its underlying parameters. They are commonly used in finance to quantify the exposure of a position with respect to these variables. The most important Greeks are:

- **Delta** (Δ): the sensitivity of the option price to a small change in the underlying spot price,

$$\Delta = \frac{\partial V}{\partial S}. \quad (2.11)$$

A call option has $\Delta \in (0, 1)$ and a put has $\Delta \in (-1, 0)$, this means that call options gain value as the underlying increases its price, while the opposite happens for put options.

- **Gamma** (Γ): the second-order sensitivity of the option price to the underlying, i.e. the rate of change of Delta,

$$\Gamma = \frac{\partial^2 V}{\partial S^2}. \quad (2.12)$$

Gamma is non-negative for long vanilla options. It is largest near the strike and short maturities.

- **Vega** ($\mathcal{V}$): the sensitivity of the option price to a change in the volatility parameter,

$$\mathcal{V} = \frac{\partial V}{\partial \sigma}. \quad (2.13)$$

Vega is largest for ATM options and long maturities. It is used in this thesis to construct vega-weighted loss functions ensuring all options have the same 'importance' when contributing to the loss regardless of its nominal value.

- **Theta** (Θ): the sensitivity to the passage of time, $\Theta = \partial V / \partial t$. Theta is typically negative for long options (the option loses value as time passes).

A key practical concern for this thesis is the *quality* of the computed Greeks. Classical methods compute Greeks by numerical bumping (bump-and-reset). Bump-and-reset methods compute the option Greeks numerically by introducing a microscopic increment (a "bump") to a single input parameter of the pricing model, re-evaluating the resulting price to calculate the finite-difference sensitivity, and subsequently resetting that parameter to its baseline state before evaluating the next risk factor. This approach is usually unreliable near the boundaries, because three slightly different 'spaces' are mixed ('before' the boundary, at the boundary and 'after' the boundary). PINNs compute Greeks as exact partial derivatives of the learned function through automatic differentiation, avoiding this issue.

2.2. Pricing Theory

2.2.1. The Black–Scholes Model and PDE

The **Black–Scholes model** [6] is the universally-used model for equity option pricing. It assumes that the underlying asset follows a **geometric Brownian motion (GBM)**:

$$dS_t = (r - q) S_t dt + \sigma S_t dW_t \quad (2.14)$$

where r is the risk-free rate, q is the continuous dividend yield, $\sigma > 0$ is the constant volatility, and W_t is a standard Brownian motion. The model is built on top of the following assumptions: markets are frictionless (no transaction costs or taxes), trading is continuous, the risk-free rate and volatility are constant, and short-selling is unrestricted.

Under these assumptions, by applying **Itô's lemma** to $V(S_t, t)$, one obtains the **Black–Scholes PDE** (see [7] for the full mathematical construction):

$$\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + (r - q) S \frac{\partial V}{\partial S} - rV = 0, \quad (2.15)$$

subject to the terminal condition $V(S, T) = \max(S - K, 0)$ for a European call. Solving this PDE analytically yields the **Black–Scholes closed formula**:

$$C^{\text{BS}} = S_0 e^{-qT} N(d_1) - K e^{-rT} N(d_2), \quad (2.16)$$

$$d_1 = \frac{\ln(S_0/K) + (r - q + \frac{1}{2}\sigma^2)T}{\sigma \sqrt{T}}, \quad d_2 = d_1 - \sigma \sqrt{T}, \quad (2.17)$$

where $N(\cdot)$ is the standard normal cumulative distribution function.

In log-moneyness coordinates $m = \ln(S_0/K)$ and time to expiry $\tau = T - t$, defining the normalised price $v = C/K$, the Black–Scholes PDE (2.15) can also be expressed as:

$$\frac{\partial v}{\partial \tau} - \frac{1}{2}\sigma^2 \frac{\partial^2 v}{\partial m^2} - \left(r - q - \frac{1}{2}\sigma^2\right) \frac{\partial v}{\partial m} + rv = 0, \quad (2.18)$$

subject to the initial condition $v(m, 0) = \max(e^m - 1, 0)$. This coordinate system is used throughout the thesis because it centres the domain around at-the-money and removes the scale dependence on the spot level.

Implied volatility. It is possible to recover the volatility from the Black-Scholes formula, this is, getting the volatility that when plugged into the formula, would reproduce the market price of a given option. This value is called the **implied volatility (IV)**:

$$\sigma_{\text{imp}}(K, T) = C^{\text{BS}, -1}(C^{\text{market}}(K, T)). \quad (2.19)$$

2.2.2. Limitations of Constant Volatility: The Empirical Smile

The Black–Scholes model assumes that implied volatility should be the same for all strikes and maturities, but this is not the reality in the markets. In practice, the implied volatility surface $\sigma_{\text{imp}}(K, T)$ is not flat.

When implied volatilities are plotted as a function of strike (or log-moneyness) for a fixed maturity, they form a curved shape known as the **volatility smile**. For equity indices, this curve is usually not symmetric: it slopes downward from low strikes to high strikes, forming what is known as "the **volatility skew**". OTM puts (low strikes, $m < 0$) carry significantly higher implied volatility than OTM calls (high strikes, $m > 0$). This shape is persistent and well-documented across all major equity markets [8].

The phenomenon has two explanations.

1. Investors are usually 'long' on the stock market and are therefore typically willing to protect their portfolio against market crashes, hence accepting to pay an 'extra' for downside protection (OTM puts), driving up their prices and their implied volatilities.
2. During market crashes, both realized volatility and correlation across stocks tend to rise sharply (contrary to GBM, which assumes that asset returns move independently and volatility is constant). Therefore, OTM puts, which act as a 'counter-measure', are inherently worth more as they provide a payout when investors need it the most.

A direct consequence of the skew is that the BS model cannot price more than one option correctly at a time: fitting it to an ATM option mismeasures OTM options, and vice versa. The need for a model that reproduces the observed smile motivates the development of local volatility models, described in the next section.

2.3. Local Volatility

The **local volatility model** [1, 9] extends the BS model by introducing a volatility surface, where volatility is no longer constant and depends on both the spot price and time: $\sigma = \sigma_{\text{loc}}(S, t)$. The equation describing the movement of the underlying now becomes:

$$dS_t = (r - q) S_t dt + \sigma_{\text{loc}}(S_t, t) S_t dW_t. \quad (2.20)$$

Bruno Dupire [1] showed that given a complete and arbitrage-free implied volatility surface, the local volatility function is uniquely determined.

2.3.1. Dupire's Formula and the Inverse Problem

Dupire [1] showed that the local volatility surface can be recovered analytically from the market prices of European call options via a formula that expresses σ_{loc} as a ratio of partial derivatives of the call price surface with respect to strike and maturity:

$$\sigma_{\text{loc}}^2(K, T) = \frac{\frac{\partial C}{\partial T}}{\frac{1}{2} K^2 \frac{\partial^2 C}{\partial K^2}}, \quad (2.21)$$

where $C(K, T)$ is the market price of a European call with strike K and maturity T .

The calibration problem, recovering the volatility surface from observed market prices, is therefore an inverse problem: given observations of v (which are noisy and discrete), find σ such that the BS PDE is satisfied.

2.3.2. Why the Inverse Problem is Ill-Posed

A problem is said to be **well-posed** in the sense of Hadamard [10] if (i) a solution exists, (ii) the solution is unique, and (iii) the solution depends continuously on the input data. The local volatility calibration problem fails the third condition and is therefore **ill-posed**.

The root cause is in the Dupire's formula (2.21): estimating the second derivative $\partial^2 C / \partial K^2$ from a finite set of discrete, noisy market quotes. A small perturbation of size ε in the prices produces amplified errors in the second-derivative estimate. This amplification even produces negative local variance in some regions ($\sigma^2 < 0$, which is physically impossible).

Two further sources of instability are:

1. **Temporal discontinuity.** Option quotes are available only at discrete maturity tenors. Estimating $\partial C / \partial T$ between slices requires interpolation, and methods that

fit each slice independently (such as SVI, described below) have no mechanism for enforcing temporal consistency, producing large jumps in $\partial C/\partial T$.

2. **Data sparsity in the wings.** Deep OTM options are illiquid and their prices have large bid-ask spreads.

2.3.3. Classical Solutions to the Ill-posed Problem

Stochastic Volatility Inspired (SVI). The SVI parametrisation fits a closed-form curve to the implied volatility smile at each individual maturity slice. The 'form' of the curve is controlled by five free parameters, and the function is optimized to fit the observed market data:

$$w(k) = a + b \left[\rho(k - m) + \sqrt{(k - m)^2 + \xi^2} \right], \quad (2.22)$$

where $w = \sigma_{\text{imp}}^2 T$ is the total implied variance, $k = \ln(K/F)$ is the log-forward moneyness, F is the forward price, and (a, b, ρ, m, ξ) are the five free parameters.

The limitation of SVI for the Dupire calibration is that it operates *slice by slice*: each maturity is fitted independently, and there is no constraint linking adjacent slices. As a result, the temporal derivative $\partial C/\partial T$ required by Equation (2.21) is estimated from the difference of two independently fitted curves ($\frac{\partial C}{\partial T} \approx \frac{C(T_2) - C(T_1)}{T_2 - T_1}$), which can differ significantly even in adjacent tenors, producing negative values of $\frac{\partial C}{\partial T}$ and therefore breaking fundamental rules such as that option prices (and total variance, w) must strictly increase with time (calendar arbitrage). In the experiments of this thesis, SVI-smoothed Dupire inversion produces undefined local variance at 65.7% of grid cells, making it unusable as a standalone calibration method.

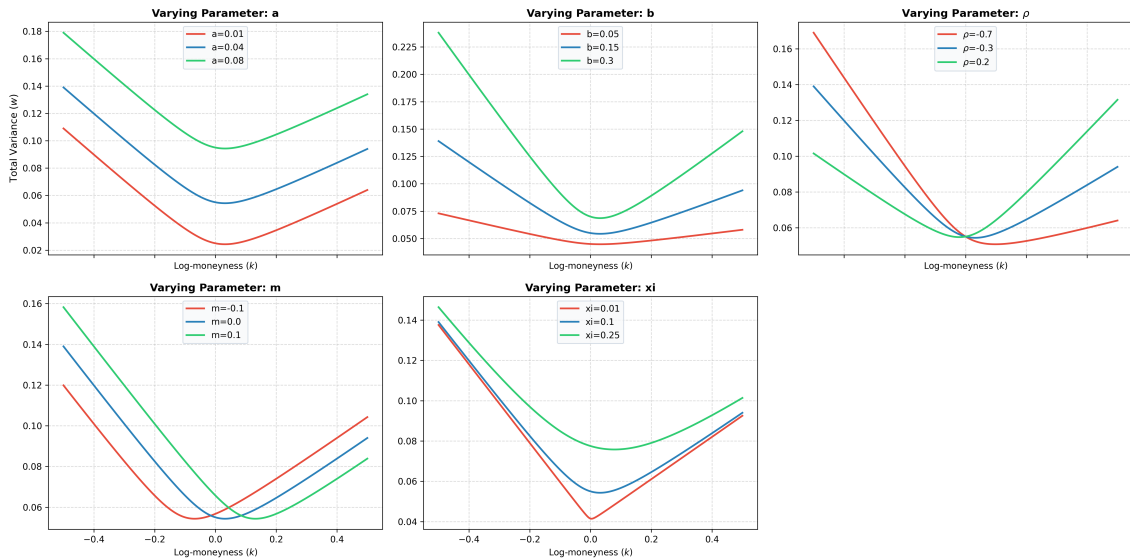


Fig. 2.1. Sensitivity analysis of the SVI parameter space. Each plot shows the impact of a single free parameter (a, b, ρ, m, ξ) on the implied variance smile $w(k)$.

Raw finite-difference Dupire. This approach consists of interpolating through the market prices with a smooth function and evaluating Equation (2.21) numerically. This method usually results in the function having to 'wobble' to reach every point, the second-derivative estimation amplifies residual noise from the interpolation, producing negative local variance in 8.5% of grid cells and volatility spikes up to $\sigma = 8.36$ in the experiments of this thesis.

Both classical methods are included in the experimental comparisons of Chapter 5 to benchmark the PINN approach.

2.4. Numerical Methods for Option Pricing

2.4.1. Finite Differences and Crank–Nicolson

Finite difference methods (FDM) solve the pricing PDE by creating a discrete grid in time and space and computing the derivatives using algebraic approximations.

Explicit, implicit, and Crank–Nicolson schemes. To generate the synthetic ground truth data in this thesis, we must choose an appropriate discretisation strategy for the time derivative. The three standard approaches are:

- **Explicit Euler:** spatial derivatives are evaluated at the *current* time level τ_n . The update is algebraically trivial (no system of equations must be solved), but the scheme is only *conditionally stable*, requiring $\Delta\tau \leq \Delta m^2 / \sigma^2$ (CFL condition).
- **Implicit Euler:** spatial derivatives are evaluated at the *next* time level τ_{n+1} , leading to a tridiagonal system of equations at each step. The scheme is *unconditionally stable* but only first-order accurate in time, $O(\Delta\tau)$.
- **Crank–Nicolson (CN):** spatial derivatives are averaged between the current and the next time level:

$$\left(I - \frac{\Delta\tau}{2} \mathcal{L}^{n+1}\right) v^{n+1} = \left(I + \frac{\Delta\tau}{2} \mathcal{L}^n\right) v^n, \quad (2.23)$$

where $\mathcal{L}$ is the discretised spatial operator. CN is unconditionally stable and second-order accurate in time and space, $O(\Delta\tau^2 + \Delta m^2)$. This balanced accuracy makes it appropriate for option pricing PDEs. Each time step requires the solution of a tridiagonal linear system, which can be solved using Thomas's algorithm.

Barrier treatment. For barrier options, the spatial domain is truncated at the barrier level m_B , and the condition $v(m_B, \tau) = 0$ is hard-coded as a boundary condition.

In this thesis, the FDM solver uses $N_m = 600$ spatial points and $N_\tau = 600$ time steps, and gives relative errors below 0.01% against the Reiner-Rubinstein closed-form solution for barriers tested under constant volatility [5] (formula explained in Annex A.0.2).

2.4.2. Monte Carlo Simulation

Monte Carlo (MC) methods price derivatives by simulating a large number of realisations of the underlying asset's path and averaging the discounted payoffs. The justification relies on the law of large numbers: as the number of simulated paths N grows, the sample mean converges to the true expected payoff.

Euler-Maruyama discretisation. Simulating the continuous-time SDE in Equation (2.20) requires a numerical time discretisation. The standard approach is the Euler-Maruyama scheme, which approximates the SDE in log-spot coordinates $X_t = \ln S_t$:

$$X_{t+\Delta t} = X_t + \left(r - q - \frac{1}{2}\sigma^2(S_t, t) \right) \Delta t + \sigma(S_t, t) \sqrt{\Delta t} Z, \quad Z \sim \mathcal{N}(0, 1). \quad (2.24)$$

Antithetic variates. This variance reduction technique consists of creating a 'mirror' path for every simulated trajectory. For every path generated using the sequence of standard normal increments Z , an antithetic path is simulated using $-Z$. This guarantees that the paths are symmetric around zero, reducing the variance of the Monte Carlo estimator.

$$\text{Var} \left(\frac{\text{Payoff}(Z) + \text{Payoff}(-Z)}{2} \right) < \frac{\text{Var}(\text{Payoff}(Z))}{2} \quad (2.25)$$

2.4.3. The BGK Continuity Correction for Barriers

A problem appears when using MC to price barrier options. A continuously monitored barrier option is knocked out the instant the spot price crosses the barrier. In a discrete simulation (as MC) with time steps Δt , the barrier is only checked at discrete times. A path that crosses the barrier *between* two steps and then recovers will not be knocked out, therefore biasing the price upward.

Broadie et al. [11] derived an explicit first-order correction that accounts for this discrete-monitoring bias. They showed that shifting the barrier by a small amount corrects the bias:

$$\tilde{B} = B \cdot \exp(\pm 0.5826 \sigma \sqrt{\Delta t}), \quad (2.26)$$

where the $+$ sign applies to a down-and-out barrier (shifted upward) and the $-$ sign to an up-and-out barrier (shifted downward). The mathematical derivation of the formula can be found at Annex A.0.3.

Without this correction, an MC simulation with $N_{\text{steps}} = 500$ systematically overprices knock-out options by 2–5%.

2.5. Neural Networks and Physics-Informed Learning

2.5.1. Neural Networks and Automatic Differentiation

A **feedforward neural network** (or multi-layer perceptron, MLP) is a parametric function $f_\theta : \mathbb{R}^{n_{\text{in}}} \rightarrow \mathbb{R}^{n_{\text{out}}}$ composed of L **layers**. Each layer applies a linear transformation followed by a nonlinear activation function ϕ :

$$\mathbf{h}^{(l)} = \phi(W^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}), \quad l = 1, \dots, L, \quad (2.27)$$

where $W^{(l)}$ and $\mathbf{b}^{(l)}$ are the weight matrix and bias vector of layer l , and $\mathbf{h}^{(0)} = \mathbf{x}$ is the input. The parameters $\theta = \{W^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^L$ are learned from data by minimising a **loss function** using gradient descent. By the **universal approximation theorem** [12, 13], a sufficiently wide single-hidden-layer network can approximate any continuous function.

Activation functions. This thesis uses tanh because the option pricing and volatility surfaces are smooth functions and the PDE loss requires computing second derivatives of the network output. Alternative choices like ReLU ($f(x) = \max(0, x)$) cannot be used, because when PyTorch's autograd computes the Black–Scholes PDE loss, the second spatial derivative ($\frac{\partial^2 v}{\partial m^2}$) is zero everywhere (and undefined at $x = 0$).

Automatic differentiation. Deep Learning libraries like PyTorch or TensorFlow compute the gradients of the network output w.r.t. any input or parameter using **automatic differentiation (AD or autograd)**. AD takes advantage of systematically applying the chain rule through the computational graph of the network, accumulating exact floating-point derivatives at each node.

The higher-order derivatives, such as $\partial^2 f / \partial x_i^2$, which corresponds to Gamma in option pricing, are computed by applying AD a second time to the first derivative. This is the mechanism by which PINNs evaluate PDE residuals during training and compute option Greeks at inference time.

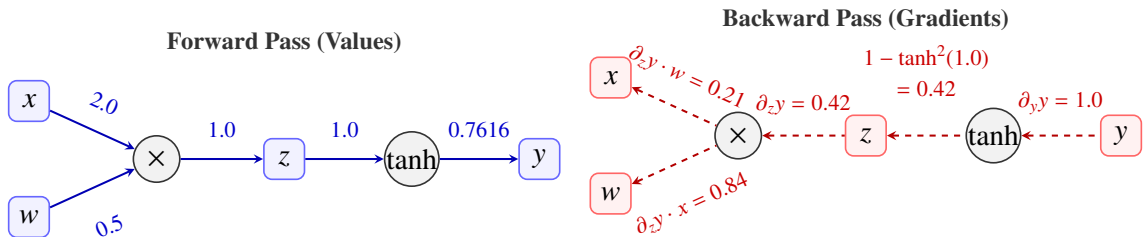


Fig. 2.2. Computational graph for $y = \tanh(w \cdot x)$. The **forward pass** (left) calculates the intermediate variables from inputs to the final output. The **backward pass** (right) applies the chain rule from right to left.

Training. The neural network is trained by minimizing a loss function using a two-stage optimization strategy. First, we use **Adam** [14], which is an optimizer more suitable for exploring the loss landscape when starting at a random starting point. Once Adam finds the general region of the solution, we switch to **L-BFGS** (Limited-memory Broyden–Fletcher–Goldfarb–Shanno), since it is more precise and converges quickly when near the optimum.

2.5.2. Physics-Informed Neural Networks

Physics-Informed Neural Networks (PINNs) [4] are neural networks that take advantage of prior physics (or financial) knowledge by incorporating the governing PDE of the system in the training loss. Therefore, the network does not learn solely from the data, but it tries to fit the data while also satisfying the underlying physics and other constraints incorporated in the loss (the PDE, the initial condition, and the boundary conditions). This is achieved through a **multi-task loss function**:

$$\mathcal{L}(\theta) = \lambda_{\text{data}} \mathcal{L}_{\text{data}} + \lambda_{\text{PDE}} \mathcal{L}_{\text{PDE}} + \lambda_{\text{IC}} \mathcal{L}_{\text{IC}} + \lambda_{\text{BC}} \mathcal{L}_{\text{BC}}, \quad (2.28)$$

where each λ is a weight controlling the relative importance of each term.

The **PDE loss** is evaluated at a set of **collocation points** $\{(m_i, \tau_i)\}_{i=1}^{N_c}$ sampled in the interior of the domain, where no data is available:

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_c} \sum_{i=1}^{N_c} \left[\frac{\partial v_\theta}{\partial \tau} - \frac{1}{2} \hat{\sigma}^2 \frac{\partial^2 v_\theta}{\partial m^2} - \left(r - q - \frac{1}{2} \hat{\sigma}^2 \right) \frac{\partial v_\theta}{\partial m} + r v_\theta \right]_{(m_i, \tau_i)}^2, \quad (2.29)$$

where v_θ and $\hat{\sigma}$ are the network outputs (price and volatility), and the derivatives are computed via autograd.

2.5.3. Hard vs Soft Boundary Enforcement

There are two strategies for enforcing boundary conditions in a PINN: using a loss penalty (soft enforcement) or via an architectural ansatz (hard enforcement).

Soft enforcement. The standard approach adds a residual term to the loss:

$$\mathcal{L}_{\text{BC}} = \frac{1}{N_b} \sum_{j=1}^{N_b} \left[v_\theta(m_j^{\text{BC}}, \tau_j) - g(m_j^{\text{BC}}, \tau_j) \right]^2, \quad (2.30)$$

where g is the true boundary value and the sum is over N_b points on the boundary. The optimiser is expected to drive this term toward zero. Soft enforcement is not suitable for constraints that *must* be strictly satisfied, as it always leaves a residual. In our case, this

residual contaminates the derivatives computed near the boundary, which are critically needed for calculating the option Greeks.

Hard enforcement via a boundary ansatz. A more robust approach is to *guarantee* the condition via a **boundary ansatz**. This involves passing the network output through a transformation that ensures the boundary condition is satisfied by construction [15]. An equivalent can be the softmax function, which hard-enforces probability constraints to be $\in (0, 1)$. For a knock-out barrier at $m = m_B$ with the condition $v(m_B, \tau) = 0$, the ansatz takes the form:

$$v(m, \tau) = g(m) \cdot \text{NN}_\theta(m, \tau), \quad (2.31)$$

where $g(m)$ is a smooth function that vanishes exactly at the barrier and approaches 1 in the interior. This thesis uses an exponential **squashed mask**:

$$g(m) = 1 - \exp(-\alpha(m - m_B)), \quad \alpha = 10, \quad (2.32)$$

for a down-and-out barrier (with the domain $m \geq m_B$). The mask equals exactly zero at $m = m_B$ and rapidly approaches 1 for interior points. Because $g(m_B) = 0$, the product in Equation (2.31) is strictly zero at the barrier for any possible network output, requiring no optimisation.

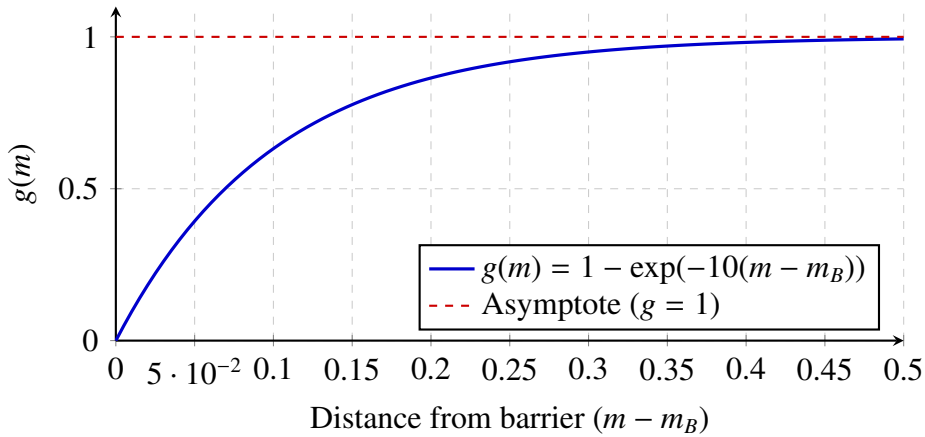


Fig. 2.3. The exponential squashed mask ansatz $g(m)$ used to hard-enforce the down-and-out barrier boundary condition.

3. RELATED WORKS

This chapter reviews the existing literature relevant to the two main aspects tackled in the thesis: the calibration of local volatility surfaces and the pricing of barrier options under non-constant volatility.

3.1. Classical Volatility Surface Calibration

The problem of recovering a volatility surface consistent with observed market prices has been studied extensively since Dupire [1] demonstrated that, given a complete and arbitrage-free continuum of European call prices $C(K, T)$, there exists a unique volatility function $\sigma_{\text{loc}}(K, T)$ that produces exactly those prices.

However, Dupire’s formula requires evaluating the ratio of partial derivatives $\partial C / \partial T$ and $\partial^2 C / \partial K^2$ from noisy and irregularly spaced market quotes. This numerical differentiation step is ill-posed because second-derivative estimation amplifies input noise, producing large oscillations and even negative local variance [16].

The most widely adopted smoothing approach to overcome this issue is the SVI parametrisation introduced by Gatheral [17] (detailed in Section 2.3.3). The extended version, known as Surface SVI (SSVI), was developed by Gatheral and Jacquier [18] to impose cross-slice consistency. Despite this, SVI-based approaches remain fundamentally *slice-by-slice*: each maturity tenor is fitted independently, lacking a built-in mechanism to enforce smoothness in the time direction. The temporal derivative required by Dupire’s formula is therefore estimated from the difference of two independently fitted curves, which can produce unrealistic results.

An alternative family of approaches relies on stochastic volatility models, which treat volatility itself as a diffusion process. The Heston model [2], with its mean-reverting square-root volatility process, is the most widely used in practice. The SABR model [3] provides a closed-form approximation for the implied smile. More recently, rough volatility models, driven by fractional Brownian motion, have been proposed [19] to better capture the short-term behaviour of the implied smile. While stochastic volatility models can reproduce the smile naturally, their calibration is highly complex, typically involving high-dimensional optimisation over a small number of global parameters.

A third classical approach addresses the ill-posedness directly through explicit regularisation (such as Tikhonov penalties on the smoothness of the surface) [20], [21]. While mathematically robust, traditional PDE-constrained optimisation is computationally expensive—a limitation that modern deep learning and Physics-Informed Neural Networks are uniquely positioned to overcome.

3.2. Neural Network Approaches to Option Pricing

The use of neural networks for derivatives pricing was first studied in the work of Hutchinson, Lo, and Poggio [22], who showed that feedforward networks could learn option pricing from historical data. This was a purely data-driven approach where the network was trained to recover the mapping from inputs to derivative values.

The topic gained interest later, when deep learning began to be used in quantitative finance. Buehler et al. [23] showed that neural networks could be used for fast derivative pricing and hedging, while Han, Jentzen, and E [24] and Sirignano and Spiliopoulos [25] demonstrated that neural methods could also be used to solve high-dimensional PDEs. This line of work led to PINN-based approaches for option pricing, where the PDE structure is built directly into the training objective.

A critical limitation of purely data-driven approaches is that they provide little structural guarantee of financial consistency, meaning the learned pricing function may violate no-arbitrage constraints or produce negative implied variances. Incorporating the PDE as a physical constraint directly addresses this vulnerability.

3.3. Physics-Informed Methods in Quantitative Finance

Physics-Informed Neural Networks (PINNs) were introduced by Raissi, Perdikaris, and Karniadakis [4] as a general framework for solving forward and inverse problems governed by partial differential equations. Initially applied to problems in fluid mechanics and heat transfer, PINNs have more recently been adopted in quantitative finance.

Forward option pricing with PINNs. A natural first application of PINNs to finance is solving the Black–Scholes PDE as a forward problem. By embedding the PDE residual in the loss, the network learns the complete price surface across strikes and maturities in a single training run, producing smooth, analytically differentiable Greeks via automatic differentiation, which is a huge advantage over bump-and-reset finite-difference methods.

Inverse calibration with PINNs. The use of PINNs to recover the volatility surface has received less attention but represents one of the most promising applications. Recovering a local volatility function from observed option prices can be model as a PINN problem, where the Dupire PDE acts as a soft constraint alongside a data-fitting loss. No relevant published study has validated this approach on real market data with a systematic comparison against classical calibration benchmarks.

Loss function design and training strategies. Training PINNs involves handling difficult multi-task loss landscapes. To address this, Wang, Teng, and Perdikaris [26] proposed adaptive loss weighting, mitigating the gradient imbalance.

Boundary enforcement. A distinction central to this thesis is between *soft* and *hard* boundary enforcement. Soft enforcement adds a boundary penalty term to the loss, while

hard enforcement modifies the network architecture so that the boundary condition is satisfied by construction (detailed in Section 2.5.3). The idea was proposed by Lagaris, Likas, and Fotiadis [15], who introduced ansatzes of the form $v(x) = A(x) + B(x) \cdot \text{NN}(x)$, where A satisfies the boundary condition and B vanishes on the boundary. In the financial context, hard enforcement has not been widely adopted, leaving the hard-constraint approach relatively unexplored for barrier options.

3.4. Barrier Option Pricing under Local Volatility

Barrier options are widely traded exotic derivatives. Under constant volatility, their pricing is well understood: closed-form solutions were derived by Reiner and Rubinstein [5] for standard single-barrier options (see Section 2.1.2 and Annex A.0.2). But under local volatility, closed-form solutions are generally unavailable. Three main approaches are established in the literature:

1. **Finite-difference methods (FDM).** The Crank–Nicolson scheme is the standard PDE solver for barrier options under local volatility (see Section 2.4). Andersen and Brotherton-Ratcliffe [27] provided a detailed treatment of this approach, while Tavella and Randall [28] discussed its practical implementation for exotic options with discrete monitoring. However, a major drawback of FDM-based barrier pricing is the computation of Greeks via numerical bumping, which often produces noisy and unreliable results near the barrier [29].
2. **Monte Carlo (MC) methods.** Monte Carlo pricing of barrier options under local volatility was studied by Glasserman [29], who identified the systematic bias arising from discrete monitoring: a simulation that checks the barrier only at the time-step endpoints misses intra-step crossings, thereby over-pricing knock-out options. Broadie, Glasserman, and Kou [11] derived a mathematical adjustment for this bias known as the BGK correction (see Section 2.4.3 and Annex A.0.3).
3. **Neural-network approaches.** The application of PINNs specifically to barrier option pricing remains largely unexplored. Standard PINN approaches using soft boundary penalties face the structural challenge of not strictly satisfying the knock-out condition $V(B, t) = 0$. Hard enforcement via a multiplicative ansatz is a natural solution; however, to the author’s best knowledge, no published study has combined a hard-boundary ansatz, a frozen calibrated local volatility surface, and a systematic comparison against both FDM and MC benchmarks for barrier options.

3.5. Research Gap and Positioning of This Thesis

The literature reviewed reveals several gaps that this thesis aims to address:

Gap 1: End-to-end pipeline from calibration to exotic pricing. Most existing studies treat calibration and pricing as separate problems. The existing PINN literature treats calibration and pricing as separate problems: calibration-focused work demonstrates volatility recovery on synthetic data but does not use the recovered surface to price exotic contracts, while pricing-focused work assumes a known volatility function and does not address the calibration step. This thesis bridges this divide by proposing a complete, unified pipeline.

Gap 2: Validation on real market data. The majority of PINN calibration studies are tested exclusively on synthetic data generated by known parametric models (such as Black–Scholes or Heston). Real market data introduces structural challenges that synthetic experiments do not capture, such as bid-ask spread noise and discrete strike-maturity grids.

Gap 3: Hard boundary enforcement for barrier PINNs. The existing literature on PINN-based option pricing uses soft boundary losses almost exclusively. For barrier options, where the boundary condition is a hard financial constraint (the contract is worthless at the barrier), soft enforcement is mathematically inadequate. This thesis introduces an architectural ansatz to satisfy the knock-out condition by construction.

In summary, this thesis is positioned at the intersection of PINN-based PDE solving, local volatility calibration, and exotic derivative pricing. Its principal contribution is the integration of these isolated components into a unified, end-to-end pipeline that is rigorously validated on real market data.

4. METHODOLOGY

The thesis is structured in four phases, where each phase builds on the previous: Phase 1 just validates that the PINN works correctly on the forward pricing problem, Phase 2 implements the calibration problem on synthetic data, Phase 3 applies it to real market data, and Phase 4 uses the calibrated surface to price barrier options.

4.1. Overall pipeline architecture

1. **Phase 1 – Forward problem.** The PINN solves the Black–Scholes PDE with known constant volatility and is validated against the closed-form solution.
2. **Phase 2 – Inverse problem.** A dual-output PINN simultaneously recovers a local volatility surface and fits option prices generated from a known parametric surface. Here the ground truth is available, so the accuracy can be measured.
3. **Phase 3 – Inverse problem.** The dual-output architecture of the previous phase is applied to real SPY option quotes, with additional adaptations for market noise and liquidity weighting.
4. **Phase 4 – Barrier option pricing.** The Phase 3 calibrated surface is frozen, and a new PINN prices knock-out barrier options using a hard boundary enforcement strategy. FDM and Monte Carlo are used as benchmarks.

The central architectural innovation is the **dual-output PINN** used in Phases 2–4. It is formed by two independent feedforward networks with shared inputs (m, τ) that produce separate outputs: a normalised call price $\hat{v}$ and a local volatility estimate $\hat{\sigma}$. The networks are only linked by the PDE residual: $\hat{\sigma}$ **must make** $\hat{v}$ **satisfy the Black–Scholes PDE** in log-moneyness form (Equation (2.18)).

4.2. Phase 1: Synthetic vanilla pricing (forward problem)

Phase 1 solves the **forward pricing problem** in a simple scenario: given constant volatility σ , price a European call by training a PINN to satisfy the Black–Scholes PDE and its initial and boundary conditions. The purpose of this phase is just to validate that the PINN architecture works and is able to get the same results as the Black–Scholes closed solution before moving to the more complex inverse problem.

4.2.1. Network architecture

The Phase 1 network is a standard MLP (Equation (2.27)) with the following characteristics:

- **Inputs:** normalised spot price $S_{\text{norm}} \in [0, 1]$ and normalised time $t_{\text{norm}} \in [0, 1]$, obtained by linearly mapping both $S \in [0, 100]$ and $t \in [0, T]$ to $[0, 1]$.
- **Hidden layers:** 4 layers of 64 neurons each, tanh activation.
- **Output:** the predicted option price $\hat{V}$.
- **Initialisation:** Xavier normal¹ for all weight matrices, zero biases.

The tanh activation is chosen because the PDE residual involves second-order derivatives of the network output, which are identically zero for piecewise-linear activations such as ReLU (explanation of how autograd works in Section 2.5.1).

Fixed training parameters of Phase 1: $K = 50.0$, $\sigma = 0.25$, $r = 0.05$, $q = 0.015$. The domain of the generated input data is $t \in [0, 0.5]$ years and $S \in [0, 100.0]$.

4.2.2. Loss function design

Following Equation (2.28), the Phase 1 loss has three terms:

$$\mathcal{L} = \lambda_{\text{pde}} \mathcal{L}_{\text{pde}} + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}} + \lambda_{\text{bc}} \mathcal{L}_{\text{bc}}. \quad (4.1)$$

- $\mathcal{L}_{\text{pde}}$: mean squared Black–Scholes PDE residual at $N_c = 10,000$ collocation points sampled in the domain interior.
- $\mathcal{L}_{\text{ic}}$: mean squared error on the terminal condition $V(S, T) = \max(S - K, 0)$ (Equation (2.1)). This is the most problematic region, as the payoff function is continuous but not differentiable at $S = K$, as we will see in Section 5.1.
- $\mathcal{L}_{\text{bc}}$: mean squared error on the boundaries:
 - If $S = 0 \Rightarrow V = 0$. A call is worthless if the underlying has no value.
 - At $S = S_{\text{max}} \Rightarrow V \approx S_{\text{max}} e^{-qT} - K e^{-rT}$. Deep ITM approximation.

No market observations are used so there is no data term in Phase 1.

¹Xavier normal initialisation draws weights from a normal distribution with mean 0 and variance $\frac{2}{n_{\text{in}} + n_{\text{out}}}$, where n_{in} and n_{out} are the number of input and output units of the layer.

4.2.3. Training procedure

Training follows the strategy described in Section 2.5.1. **Phase A (Adam)**: 8,000 epochs, learning rate 10^{-3} with cosine annealing to 10^{-5} . Collocation points are re-sampled at each step. **Phase B (L-BFGS)**: up to 2,000 iterations, Strong Wolfe line search, and a fixed set of collocation points, which is required for L-BFGS (this is discussed in Section 4.3.3).

The trained model is validated against the Black–Scholes closed-form solution (Equation (2.16)). Numerical results are reported in Section 5.1.

4.3. Phase 2: Synthetic local volatility calibration (inverse problem)

Phase 2 introduces the **inverse problem**: recovering the local volatility surface from option prices. A custom-generated dataset is used so recovery accuracy is measurable. This phase serves as a preliminary step before applying the method to real market data.

4.3.1. The dual-output PINN

The Phase 1 architecture has to be adapted to both fit option prices and recover the implicit volatility, but just adding another loss term turned out not to be a good approach, because the strong gradient signal from the data loss dominates and prevents the vol output from learning the surface. The solution is to use **two independent networks**. Both networks receive the same input: $(m_{\text{norm}}, \tau_{\text{norm}}) \in [0, 1]^2$ and only interact through the PDE loss:

- **Price network**: 4 hidden layers $\times$ 64 neurons, tanh, Xavier init. Outputs normalised call price $\hat{v}$.
- **Vol network**: 3 hidden layers $\times$ 64 neurons, tanh, Xavier init. The output is passed through softplus and a minimum floor $\sigma_{\min} = 0.01$ is added, to ensure $\hat{\sigma} > 0$:

$$\hat{\sigma} = \text{softplus}(\text{raw}) + \sigma_{\min}. \quad (4.2)$$

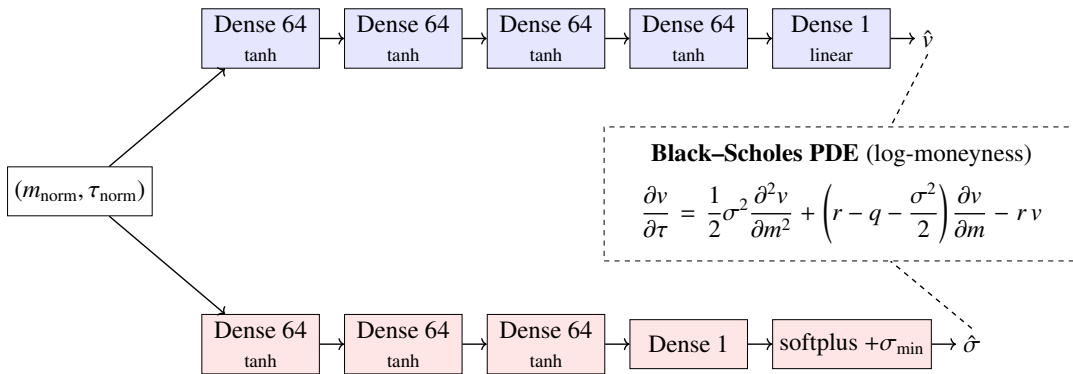


Fig. 4.1. Architecture of the coupled PINN model.

4.3.2. Synthetic local volatility surface

The ground-truth surface used to generate synthetic training data is a custom function:

$$\sigma_{\text{true}}(m, \tau) = 0.2 + 0.1 \exp\left(-\frac{m^2}{0.3}\right) + 0.05 \tau. \quad (4.3)$$

This surface has three components: a base level of 20%, an ATM bump of 10% that decays symmetrically with moneyness (producing a symmetric smile), and a positive term structure slope of 5% per year.

Synthetic option prices are generated using a Crank–Nicolson FDM solver (Section 2.4.1) on a grid of 25 strikes $\times$ 10 maturities ($m \in [-0.3, 0.3]$, $\tau \in [0.08, 1.0]$), yielding $N = 250$ normalised call prices v_{FDM} that act as the observed data.

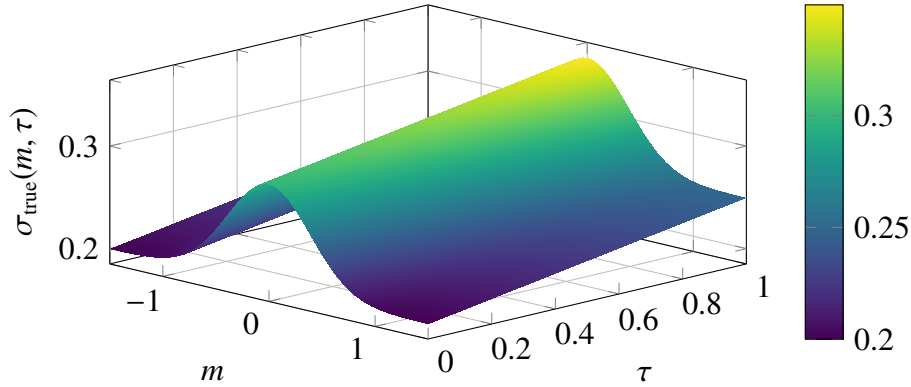


Fig. 4.2. Synthetic local volatility surface used to generate the training data.

4.3.3. Loss function and architectural modifications

The Phase 2 loss extends the general PINN formulation to include a data term, a Tikhonov smoothness regulariser, and arbitrage-free constraints:

$$\mathcal{L} = \lambda_{\text{data}} \mathcal{L}_{\text{data}} + \lambda_{\text{pde}} \mathcal{L}_{\text{pde}} + \lambda_{\text{smooth}} \mathcal{L}_{\text{smooth}} + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}} + \lambda_{\text{arb}} \mathcal{L}_{\text{arb}}, \quad (4.4)$$

with weights $\lambda_{\text{data}} = 10$, $\lambda_{\text{pde}} = 5$, $\lambda_{\text{smooth}} = 0.001$, $\lambda_{\text{ic}} = 3$, $\lambda_{\text{arb}} = 5$.

1. Second-derivative Tikhonov regularisation. The smoothness term penalises the *curvature* (second derivatives) of the recovered volatility surface, rather than its slope:

$$\mathcal{L}_{\text{smooth}} = \frac{1}{N_c} \sum_{i=1}^{N_c} (w_m \hat{\sigma}_{mm}^2 + w_\tau \hat{\sigma}_{\tau\tau}^2), \quad (4.5)$$

with $w_m = 0.01$ and $w_\tau = 1.0$ (anisotropic weights). Penalising the *first* derivative would tend to suppress volatility smiles and skews, whereas penalising the second derivative allows smooth variations while discarding spikes and oscillations. The anisotropic weights

reflect the fact that volatility smiles are naturally more curved across log-moneyness m than across maturity τ (curvature in the m -direction is penalised 100 times less than curvature in the maturity direction).

2. C^2 -continuous arbitrage-free constraints. Without soft penalties on no-arbitrage principles, the price network can fit the 250 training points perfectly while predicting unphysical values between them.

The Dupire calendar constraint for European options with dividend yield q is (derived from Equation (2.21)):

$$v_\tau + r v - (r - q) v_m \geq 0. \quad (4.6)$$

This is more general than $v_\tau \geq 0$, which may not hold for deep ITM calls with high dividends. The butterfly constraint requires convexity across strikes:

$$v_{mm} - v_m \geq 0. \quad (4.7)$$

Both are penalised using a *cubed* ReLU:

$$\mathcal{L}_{\text{arb}} = \mathbb{E}[\max(0, -\text{cal})^3] + \mathbb{E}[\max(0, -\text{btf})^3], \quad (4.8)$$

where “cal” and “btf” denote the left-hand sides of Equations (4.6) and (4.7). The cubic exponent is a choice made to solve an error that appeared with the L-BFGS optimiser: it makes the second derivative of the penalty continuous at the boundary of the constraint. This prevents the L-BFGS optimiser from seeing undefined second derivatives, a failure that causes it to abort the training process.

3. Balanced loss weights. A data-to-PDE weight ratio of 100:1 was found to let the price network prioritise data fitting at the expense of clean second derivatives. Since the vol network learns exclusively through the PDE residual, it received noisy v_{mm} signals and did not reach good solutions. Reducing the ratio to 10:5 forces the price network to produce smooth derivatives, which the vol network can interpret better.

4. Volatility network pre-training. Before the main training, the vol network alone is trained for 500 epochs to predict a flat surface $\hat{\sigma} = 0.30$ (a reasonable prior for options). This avoids the irregular starting state produced by Xavier initialisation, from which the optimiser can get trapped in a local minimum.

5. Frozen collocation points for L-BFGS. During Phase B, interior collocation points are sampled *once* before the optimisation loop and kept fixed. When points are re-sampled between calls (as they are during the stochastic Adam phase), the loss landscape shifts between evaluations, and L-BFGS aborts its line search after very few steps. Freezing the grid allows L-BFGS to run thousands of iterations.

4.3.4. Training procedure

Phase A (Adam): 15,000 epochs, $\text{lr} = 10^{-3}$ with cosine annealing to 10^{-5} . After 500 pre-training epochs for the vol network, the full five-term loss (Equation (4.4)) is minimised.

Phase B (L-BFGS): up to 5,000 iterations, Strong Wolfe line search, frozen collocation grid of $N_c = 2,000$ PDE points.

4.4. Phase 3: Real-data calibration on SPY

Phase 3 applies the dual-output PINN to real SPY option quotes. The architecture is the same as in Phase 2, but the data preprocessing pipeline, the data loss, and the training domain are adapted to the characteristics of the real data.

4.4.1. Data source and preprocessing

The dataset consists of SPY end-of-day option mid-prices on 20 September 2021 (SPY at \$434.01, VIX $\approx$ 20.9). The data is filtered to retain:

- **OTM options only:** moneyness $K/S_0 \in [0.80, 1.20]$. SPY options are American-style, but the early-exercise premium is negligible for OTM contracts (Section 2.1.1).
- **Liquid maturities:** days to expiry $\text{DTE} \in [7, 365]$, excluding very short-dated options (that are usually at the non-differentiable spot of $S = K$) and very long-dated contracts (poor liquidity).

After filtering, $N = 1,956$ option quotes remain, with both calls ($m > 0$) and puts ($m < 0$). All OTM puts were converted to call-equivalent prices using put–call parity (Equation (2.4)).

The risk-free rate r is linearly interpolated from the US Treasury yield curve at the option’s maturity. The dividend yield is set to a constant $q = 1.5\%$, estimated from the S&P 500 dividend yield. The training domain is $m \in [-0.25, 0.22]$ and $\tau \in [0, 1.0]$.

4.4.2. Vega-weighted data loss

A simple MSE data loss treats all options equally, and it was found that this is not desirable. First, ATM options have the largest vega² and dominate the loss, so the recovered volatility surface may fit ATM prices well while forgetting about the wings. Second, deep

²A small change in volatility produces a large change in price for ATM contracts. As a result, ATM prices contain a lot of information about volatility and the optimiser can reduce the loss heavily by fitting ATM options.

OTM short-dated contracts (which tend to have small vega) have very small normalised prices and their absolute errors may be negligible.

The data loss applies inverse-vega weights to overcome both issues:

$$\mathcal{L}_{\text{data}} = \frac{1}{N} \sum_{i=1}^N w_i (\hat{v}_i - v_i^{\text{mkt}})^2, \quad w_i \propto \frac{1}{\mathcal{V}_i + \varepsilon}, \quad (4.9)$$

where $\mathcal{V}_i$ is the Black–Scholes vega of option i (Equation (2.13)). Weights are capped at the 95th percentile to prevent near-zero vegas from producing extreme values, and then normalised so that $\bar{w} = 1$. This weighting ensures OTM options (illiquid and harder-to-fit) receive enough attention.

4.4.3. Full loss function and training

The Phase 3 loss has the same five terms as Phase 2 (Equation (4.4)), with two adaptations:

- The data loss uses the inverse-vega weights of Equation (4.9).
- The arbitrage penalty is lowered to $\lambda_{\text{arb}} = 1.0$ (vs. 5.0 in Phase 2). Real market data contains some arbitrage violations that are justified by bid-ask noise or liquidity and should not be penalised harshly. The true Dupire calendar constraint (Equation (4.6)) and butterfly constraint (Equation (4.7)) are unchanged.
- The remaining weights are: $\lambda_{\text{data}} = 10$, $\lambda_{\text{pde}} = 5$, $\lambda_{\text{smooth}} = 0.001$, $\lambda_{\text{ic}} = 3$.
- The anisotropic Tikhonov weights are still: $w_m = 0.01$, $w_\tau = 1.0$.

The vol network is pre-trained for 500 epochs to $\hat{\sigma} = 0.20$ (the approximate mean market vol on the chosen date). Training: **Adam 15,000 epochs** ($\text{lr} = 10^{-3}$, cosine annealing) followed by **L-BFGS up to 5,000 iterations** (Strong Wolfe, frozen collocation grid of $N_c = 10,000$ PDE points).

4.4.4. Benchmark methods: raw FDM Dupire and SVI-smoothed Dupire

Two classical alternatives are implemented to benchmark against the PINN.

Raw FDM Dupire: the observed implied volatilities are interpolated onto a regular (m, τ) grid using bilinear interpolation³, and Dupire’s formula (Equation (2.21)) is applied numerically via second-order finite differences. No smoothing is applied before differentiation.

SVI-smoothed Dupire: the five-parameter SVI model (Equation (2.22)) is fitted independently to each maturity slice. The temporal derivative $\partial C / \partial T$ required by Dupire is

³Bilinear interpolation estimates the value at a point located between four neighbouring grid nodes by performing linear interpolation in one axis and then in the other.

then estimated as a finite difference between adjacent SVI curves. Despite the smoothing within each slice, the cross-slice temporal derivative remains unstable⁴ because the two SVI fits are independent (2.3.3).

Both benchmarks use the same domain grid as the PINN and are evaluated at the same set of test points. The comparison is presented in Section 5.3.

4.5. Phase 4: Barrier option pricing

Phase 4 uses the Phase 3 local volatility surface (vol network weights are not updated during Phase 4 training) to price knock-out barrier options. Three independent pricing methods are cross-validated: a CN FDM solver, a MC simulator, and a PINN.

4.5.1. Problem formulation and coordinate bridge

Barrier option pricing in Phase 4 is a **forward problem**: $\hat{\sigma}$ is known and the goal is to solve for the option price $v(m, \tau)$ while satisfying the Dupire PDE and the boundary condition $v(m_B, \tau) = 0$.

A conflict arises between the calibrated volatility surface (Phase 3) and the exotic option pricer (Phase 4) from the different definitions of log-moneyness.

Phase 3 is built on Dupire's *forward* equation. It fixes the initial stock price (S_0), and then defines moneyness across all hypothetical strikes (K_{Dupire}). Therefore, the coordinate is $m_{\text{Dupire}} = \ln(S_0/K_{\text{Dupire}})$.

On the other hand, Phase 4 solves the Black–Scholes *backward* PDE to price an option. Here, the strike (K) is fixed, and the PDE evaluates across all moving, future stock prices (S). The spatial coordinate is $m_{\text{grid}} = \ln(S/K)$.

When the Phase 4 NN evaluates a grid point at a future spot price S , it must look in the frozen Phase 3 surface for the local volatility at that point. By the definition of local volatility, **the volatility at a future spot S corresponds exactly to the implied variance of an option whose strike is S** ($K_{\text{Dupire}} = S$). Therefore, the coordinate inserted into Phase 3 is $\ln(S_0/S)$.

Rather than recalculating logarithms at every collocation point, this query coordinate can be expressed as the difference between the fixed moneyness of the option being priced (m_{spot} , a constant for the given contract) and the moving grid coordinate (m_{grid}):

$$\ln\left(\frac{S_0}{S}\right) = \ln\left(\frac{S_0}{K}\right) - \ln\left(\frac{S}{K}\right) = m_{\text{spot}} - m_{\text{grid}}. \quad (4.10)$$

⁴Although each maturity slice is smoothed by an independent SVI fit, the temporal derivative $\partial C/\partial T$ is approximated using finite differences between adjacent slices. Small errors between two SVI smiles can be amplified when computing the difference quotient $(C(T + \Delta T) - C(T))/\Delta T$.

4.5.2. Crank–Nicolson FDM and Monte Carlo benchmarks

The FDM solver (Section 2.4.1) is extended to barrier options by truncating the spatial domain at the barrier level and imposing $v(m_B, \tau) = 0$. The grid uses $N_m = 600$ spatial points and $N_\tau = 600$ time steps. The barrier is placed at the lower boundary for down-and-out contracts and at the upper boundary for up-and-out contracts.

The Monte Carlo engine (Section 2.4.2) simulates asset paths following the calibrated local volatility with the Euler–Maruyama formula (Equation (2.24)) with $N_{\text{paths}} = 100,000$ and $N_{\text{steps}} = 500$. Antithetic variates are used for variance reduction. To correct for discrete monitoring bias, the BGK barrier shift (Equation (2.26)) is applied.

Both engines are first validated against the Reiner–Rubinstein closed-form solution (Section 2.1.2) under constant volatility before being tested with the calibrated LV surface: the FDM solver reproduces the closed form to within 0.14%, and the Monte Carlo estimator agrees with it to within one standard error.

4.5.3. Barrier PINN with squashed boundary ansatz

The barrier PINN uses the hard boundary enforcement strategy mentioned in Section 2.5.3. The squashed mask ansatz (Equation (2.31)) guarantees $v(m_B, \tau) = 0$ always:

$$v(m, \tau) = g(m) \cdot \text{NN}_\theta(m_{\text{norm}}, \tau_{\text{norm}}), \quad (4.11)$$

where $g(m) = 1 - \exp(-\alpha(m - m_B))$ with $\alpha = 10$ for a down-and-out barrier (domain $m \geq m_B$). For an up-and-out barrier (domain $m \leq m_B$), the mask is reversed: $g(m) = 1 - \exp(-\alpha(m_B - m))$. The network is formed by 4 hidden layers $\times$ 64 neurons, tanh activation, Xavier init.

There is no data loss term (no barrier option prices are available). The loss contains only two terms:

$$\mathcal{L} = \lambda_{\text{pde}} \mathcal{L}_{\text{pde}} + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}}, \quad (4.12)$$

with $\lambda_{\text{pde}} = 1$ and $\lambda_{\text{ic}} = 10$. The PDE residual is computed at $N_c = 2,000$ interior collocation points using the frozen vol surface (via Equation (4.10)); the IC loss enforces $v(m, 0) = \max(e^m - 1, 0)$ at $N_{\text{ic}} = 500$ boundary points.

Training uses Adam for 5,000 epochs with cosine annealing ($\text{lr} = 10^{-3}$). L-BFGS is not used here because without a data term, the PDE loss landscape is already smooth and Adam can converge alone. One separate model is trained per barrier configuration (barrier level $\times$ barrier type), given that the mask $g(m)$ and spatial domain depend on m_B .

Once trained, we can easily obtain Delta and Gamma via autograd. The smoothness of PINN Greeks near the barrier, compared to FDM bump-and-reset, is very noticeable and is one of the key comparisons in Chapter 5.

5. RESULTS

This chapter presents the results for each phase of the pipeline.

5.1. Phase 1: Synthetic vanilla pricing

The Phase 1 PINN is validated against the Black–Scholes closed-form solution (Equation (2.16)) on a grid of 200×200 points over the full domain. Table 5.1 summarises the metrics.

Metric	Value
RMSE	0.0023
MAE	0.0014
Max absolute error	0.1368

TABLE 5.1. PHASE 1 ERROR METRICS VS. BLACK–SCHOLES CLOSED-FORM SOLUTION

The RMSE and MAE show that the PINN correctly solves the Black–Scholes PDE and that autograd computes the required second derivatives accurately. The maximum error of 0.1368 is an outlier that has an explanation, it is localised at $S = K = 50$ at expiry $t = T$. At this point the true payoff function $\max(S - K, 0)$ has a *kink*. Because tanh activations produce smooth functions, the PINN cannot reproduce the kink. Moving away from expiry, the Black–Scholes diffusion term $\frac{1}{2}\sigma^2 S^2 \partial^2 V / \partial S^2$ smooths the payoff, and the error goes to zero. This is visible in the error heatmap (Figure 5.1): the only error is located at the expiry boundary.

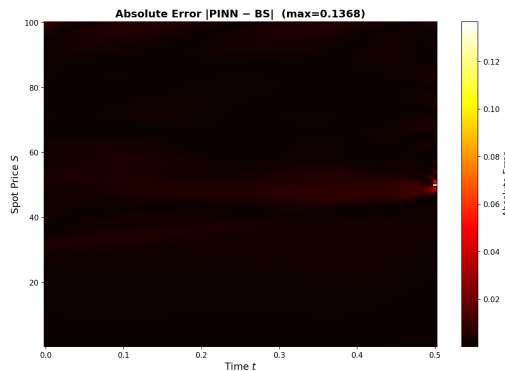


Fig. 5.1. Absolute error $|\hat{V} - V_{BS}|$ over the (S, t) domain. The spike at $S = K, t = T$ is the kink.

Figure 5.2 shows price slices at four different times.

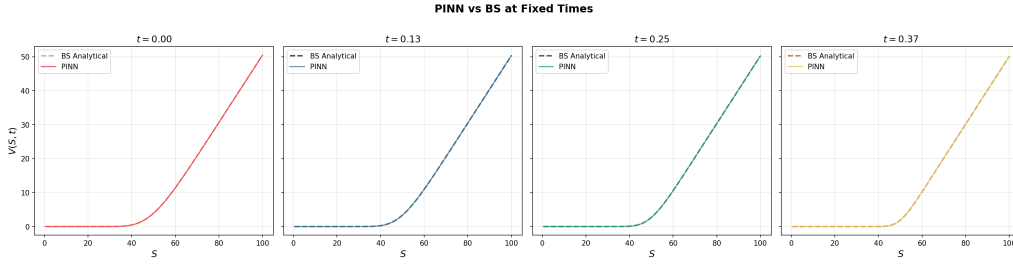


Fig. 5.2. PINN vs. Black–Scholes price slices at four time points. The curves overlap except at $t = T$ near $S = K$.

These results confirm that the PINN can perfectly reproduce the analytical solution.

5.2. Phase 2: Synthetic LV calibration

The dual-output PINN is trained on the 250 synthetic prices generated from the ground-truth surface σ_{true} (Equation (4.3)). After training, the recovered volatility surface $\hat{\sigma}(m, \tau)$ is evaluated at the same points and compared to σ_{true} .

Exactly 87.5% of grid cells fall within 5% relative error of the true surface, and 99.5% fall within 10%. The recovered surface correctly identifies the symmetric ATM bump and the positive term structure slope. The main residual error concentrates at the edges of the domain ($|m| \approx 0.3$, short maturities $\tau \approx 0.08$), where the training data is sparse and the PDE constraint is less informative.

Figure 5.3 compares the true and recovered surfaces; Figure 5.4 shows smile cross-sections at two maturities.

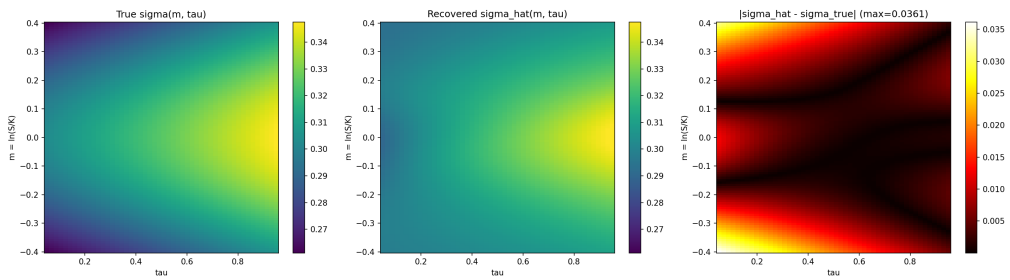


Fig. 5.3. True (left) vs. PINN-recovered (right) local volatility surface $\sigma(m, \tau)$.

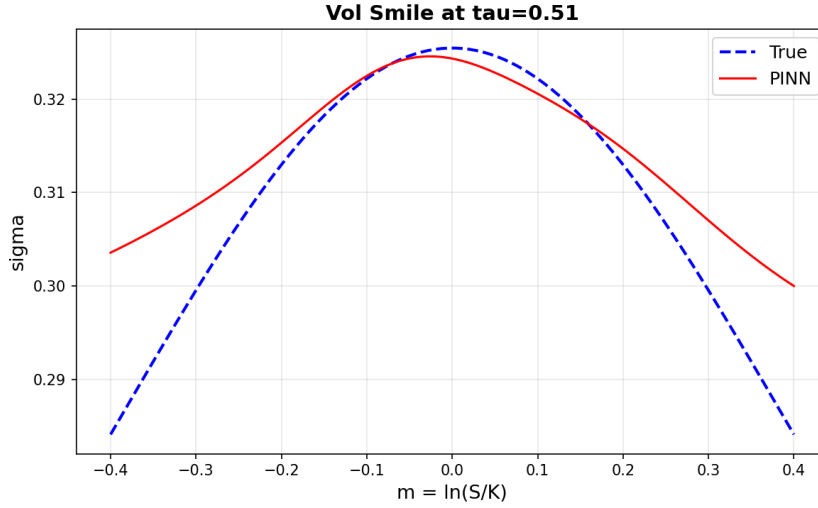


Fig. 5.4. Smile cross-sections at $\tau = 0.25$ and $\tau = 0.75$. The recovered $\hat{\sigma}$ (dashed) closely follows σ_{true} (solid).

The Phase 2 results demonstrate that the dual-output architecture and the five loss modifications (Section 4.3.3) together succeed in recovering a local volatility surface from synthetic option prices.

5.3. Phase 3: Real-data calibration on SPY

5.3.1. Calibration accuracy

After training (Adam 15,000 epochs + L-BFGS 5,000 iterations), the PINN price network achieves $R^2 = 0.9991$ on the 1,956 SPY market quotes. Figure 5.5 shows the predicted vs. observed normalised prices $\hat{v}$ vs. v^{mkt} . The fit is tight across the entire moneyness range, with no visible bias.

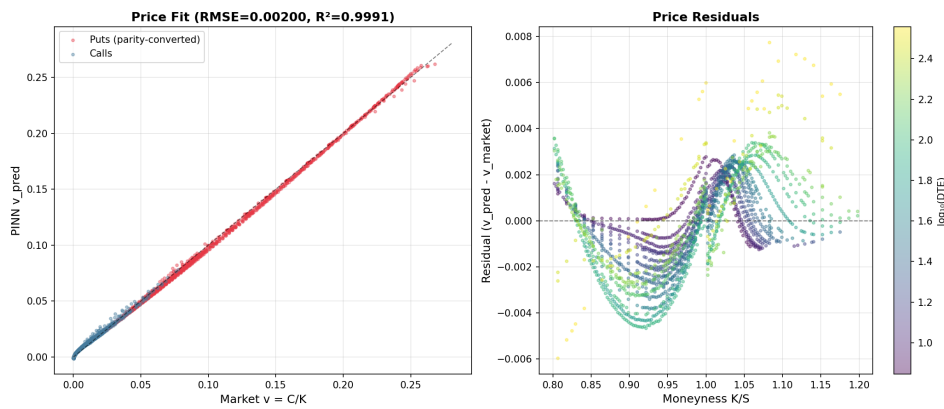


Fig. 5.5. Predicted vs. observed normalised call prices on the SPY dataset ($R^2 = 0.9991$).

5.3.2. Recovered local volatility surface

The recovered surface $\hat{\sigma}(m, \tau)$ is shown in Figure 5.6. It exhibits the expected SPY characteristics: a pronounced negative skew (higher vol for OTM puts, $m < 0$) reflecting the equity risk premium, and a mild upward term structure (long-dated options carry slightly higher vol than short-dated ones). The surface is smooth with $\hat{\sigma} \in [0.08, 0.36]$ across the domain, what is reasonable for SPY in September 2021 (VIX ≈ 20.9).

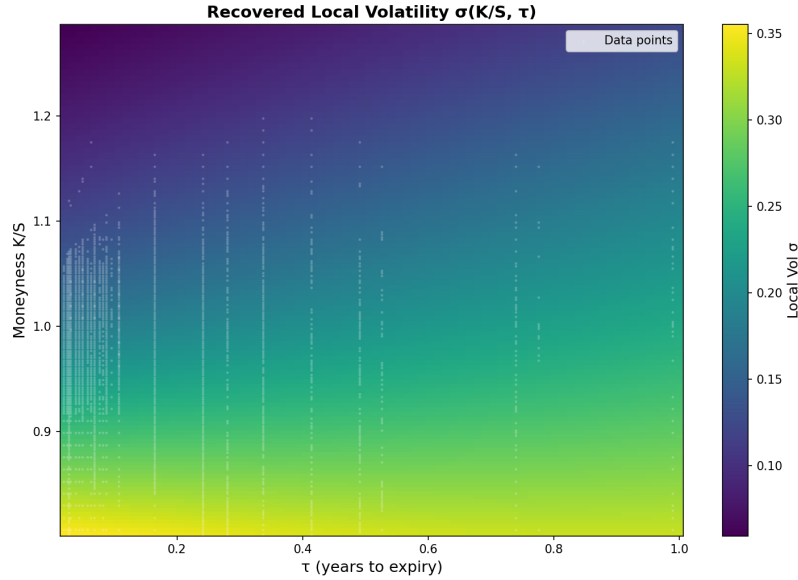


Fig. 5.6. Recovered local volatility surface $\hat{\sigma}(m, \tau)$ from SPY options on 20 September 2021.

5.3.3. Three-way comparison: raw FDM vs SVI-smoothed vs PINN

Table 5.2 summarises the surface validity of the three methods across the same 96×100 evaluation grid (9,600 cells). A cell is considered *valid* if the inferred local variance is strictly positive (real, non-imaginary, non-zero volatility).

Method	Valid cells	Negative variance	NaN cells	$\hat{\sigma}$ range
Raw FDM Dupire	91.5%	8.5% (814/9600)	0%	[0.01, 8.36]
SVI-smoothed Dupire	34.3%	0.7% (64/9600)	65.7%	[0.003, 1.26]
PINN	100.0%	0.0%	0%	[0.08, 0.36]

TABLE 5.2. PHASE 3: SURFACE VALIDITY COMPARISON
ACROSS METHODS

There are big differences. The raw FDM Dupire inversion produces negative local variance at 8.5% of cells and also outputs an unreasonable maximum of $\hat{\sigma} = 8.36$ (836% annualised vol). SVI fails at the cross-slice temporal derivative, 65.7% of grid cells are NaN because independent slice-fitting allows the curves to cross. This calendar arbitrage

produces a negative numerator ($\frac{\partial C}{\partial T} < 0$) in the Dupire equation, resulting in undefined (imaginary) volatility. Only the PINN produces a surface that is 100% valid.

Figure 5.7 shows the three-panel heatmap; Figure 5.8 shows smile cross-sections at four maturities ranging from 11 to 271 days to expiry.

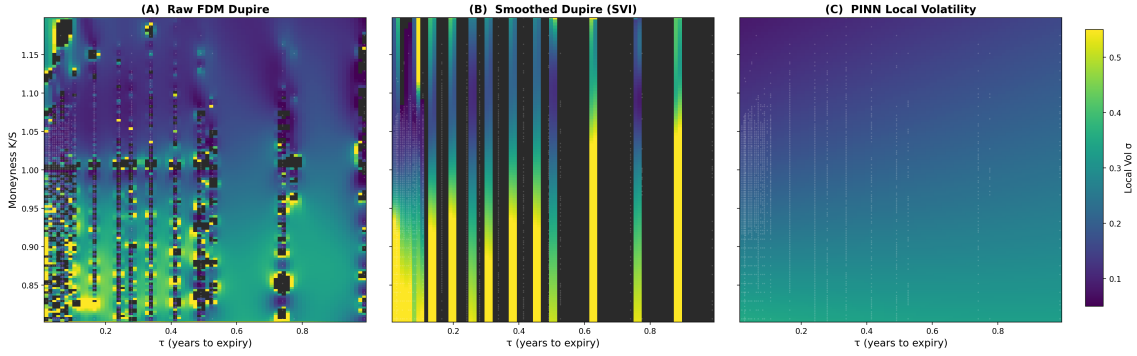


Fig. 5.7. Local volatility surfaces recovered by (A) raw FDM Dupire, (B) SVI-smoothed Dupire, and (C) the PINN. Dark patches in (A) are imaginary-variance cells; large dark regions in (B) are NaN. The PINN surface (C) is smooth and fully defined.

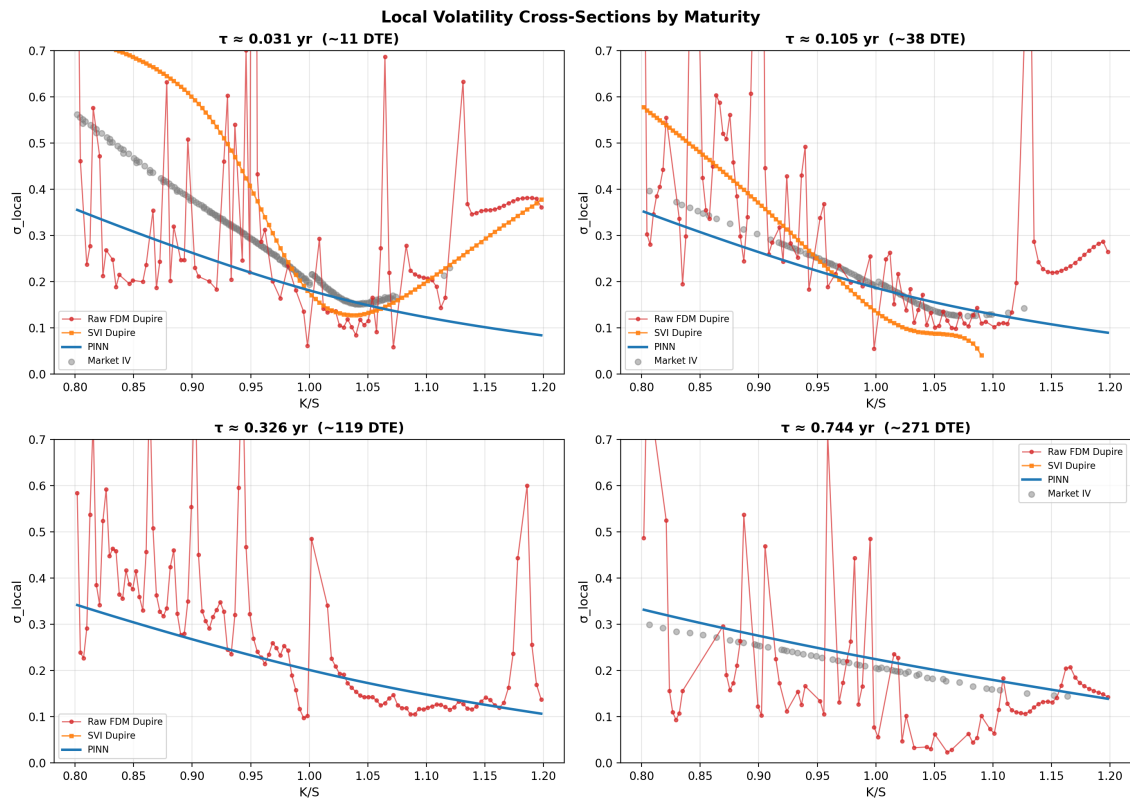


Fig. 5.8. Local volatility smile cross-sections at $\tau \approx 0.031$ (11 DTE), $\tau \approx 0.105$ (38 DTE), $\tau \approx 0.326$ (119 DTE), and $\tau \approx 0.744$ (271 DTE). Gray dots are market implied vols; lines are method outputs (they are missing in two of the plots because there was no market data at that exact expiration date). SVI curves are absent where the method returned NaN.

5.4. Phase 4: Barrier option pricing

5.4.1. Four-way pricing comparison

Six barrier option configurations are priced: three down-and-out calls (DOC) and three up-and-out calls (UOC), each at $S_0 = K = \$434$ and $T = 0.5$ years, with three barrier levels per type. Table 5.3 shows the prices produced by the four engines — constant-vol Black–Scholes as a reference, the two numerical benchmarks (FDM-LV and MC-LV), and the barrier PINN — alongside the LV correction relative to BS (“Diff%”) and the PINN’s deviation from the FDM benchmark (“PINN err%”).

Before being applied to the calibrated LV surface, all three engines were validated against the Reiner–Rubinstein closed-form solution under constant volatility ($\sigma = 0.20$), on the same six configurations and over the same computational domain, maturity and training budget used below. The FDM engine reproduces the closed form to within 0.14% in every case, and to within 0.006% on the three down-and-out configurations. The Monte Carlo engine agrees with it to within one standard error on all six configurations, which is the appropriate criterion for a stochastic estimator rather than a fixed relative tolerance. The barrier PINN reproduces it to within 1.7% on the down-and-out configurations and 5.1% on the up-and-out configurations.

Barrier B	BS-const	FDM-LV	MC-LV ($\pm$ std)	PINN-LV	Diff%	PINN err%
<i>Down-and-out call (DOC)</i>						
\$368.9	22.8298	21.8882	21.8544 ± 0.0912	21.8830	−4.1%	−0.02%
\$390.6	21.2811	19.9679	19.9099 ± 0.0913	20.0127	−6.2%	+0.22%
\$412.3	15.0528	14.1580	14.1864 ± 0.0854	13.8522	−5.9%	−2.16%
<i>Up-and-out call (UOC)</i>						
\$455.7	0.1057	0.1680	0.1649 ± 0.0043	0.5990	+59.0%	+256.5%
\$477.4	1.2407	2.4247	2.4172 ± 0.0225	3.1361	+95.4%	+29.3%
\$499.1	4.2140	8.8216	8.7685 ± 0.0489	8.5143	+109.3%	−3.5%

Diff% = $(V_{\text{FDM-LV}} - V_{\text{BS}})/V_{\text{BS}} \times 100$; PINN err% = $(V_{\text{PINN-LV}} - V_{\text{FDM-LV}})/V_{\text{FDM-LV}} \times 100$.

TABLE 5.3. PHASE 4: FOUR-WAY BARRIER OPTION PRICE
COMPARISON ($S_0 = K = \$434$, $T = 0.5$ YR)

The first conclusion we can observe is that **FDM-LV and MC-LV agree within one Monte Carlo standard error⁵ for all six configurations**, confirming that the two independent engines are consistent and that the calibrated LV surface does not produce

⁵The Monte Carlo standard error is the estimated uncertainty of a Monte Carlo price due to the finite number of simulated paths. It is computed as $\text{SE} = s/\sqrt{N}$, where s is the sample standard deviation of the discounted payoffs and N is the number of simulations. Two prices that differ by less than one standard error are statistically indistinguishable at the level of Monte Carlo sampling noise.

problems.

The barrier PINN reproduces the numerical benchmarks for down-and-out calls, but not for up-and-out calls. Its price lies within 2.2% of the FDM benchmark for all three DOC configurations (-0.02% , $+0.22\%$ and -2.16%), which is the regime in which the method can be regarded as validated against the benchmarks. For the UOC configurations the agreement degrades sharply and without a monotonic pattern in the barrier level: -3.5% at $B = \$499.1$, $+29.3\%$ at $B = \$477.4$, and $+256.5\%$ at $B = \$455.7$.

The training diagnostics locate the problem in the network rather than in a discretisation or sampling artefact of the benchmarks. The final training loss of the three DOC networks is of order 10^{-4} (2.6×10^{-4} , 2.3×10^{-4} and 4.5×10^{-4}) and decreases monotonically, whereas the three UOC networks terminate an order of magnitude higher (1.2×10^{-3} , 2.7×10^{-3} and 3.4×10^{-3}) and all three display a non-monotonic loss over their final 2,000 epochs. Repeated training confirms the instability: an independent run of the same code and hyperparameters prices $B = \$477.4$ at -3.4% relative to FDM rather than $+29.3\%$. The UOC results are therefore not reproducible at the precision that the DOC results are, and should not be read as a converged estimate of the method’s accuracy in that regime.

A control experiment isolates the cause. Repeating the six configurations under a constant volatility $\sigma = 0.20$ — identical geometry, domain, maturity and training budget, so that only the coefficient of the PDE changes and an exact reference exists — the barrier PINN reproduces the closed-form solution to within 1.7% on the down-and-out configurations and 5.1% on the up-and-out ones. This includes the $B = \$455.7$ contract, which under constant volatility is worth only \$0.106, less than its value under the calibrated surface, and is nonetheless priced to within 3.6%. The low value of the contract is therefore not the explanation for the $+256.5\%$ deviation.

What changes between the two settings is the absolute error. The three UOC configurations commit errors of \$0.004, \$0.064 and \$0.183 under constant volatility, against \$0.43, \$0.71 and \$0.31 under the calibrated surface: an increase of up to two orders of magnitude. Their final training losses, however, are almost unchanged (9.6×10^{-4} , 1.7×10^{-3} and 4.2×10^{-3} against 1.2×10^{-3} , 2.7×10^{-3} and 3.4×10^{-3}). The network therefore converges to a comparable PDE residual while pricing far less accurately once the coefficient of the equation is the calibrated surface rather than a constant, which indicates that the residual is a poor proxy for pricing accuracy in this regime.

The convergence asymmetry between the two barrier types does survive the control: the up-and-out losses remain an order of magnitude above the down-and-out ones (10^{-3} against 10^{-4}) even under constant volatility. This points to the hard ansatz $g(m) = 1 - e^{-\alpha(m_B - m)}$, which is steepest exactly where the up-and-out evaluation points lie, as the underlying difficulty, with the calibrated surface amplifying it into a pricing failure rather than causing it.

5.4.2. LV correction over Black-Scholes

The Diff% column reveals the economic impact of using a proper local volatility surface instead of constant volatility.

Down-and-out calls are priced 4–6% lower under LV than under constant vol. This can be explained by the fact that the calibrated surface assigns higher volatility to the OTM put region near the barrier ($m < 0$), making down-crossings more likely and thus reducing the DOC value.

Up-and-out calls are priced 59–109% higher under LV. In this case, the surface assigns lower volatility to the OTM call region near the barrier ($m > 0$), making up-crossings less likely, so it is worth more. The magnitude of the correction grows as the barrier moves deeper OTM (from +59% at $B = \$455.7$ to +109% at $B = \$499.1$), because these contracts are extremely sensitive to the precise shape of the smile at the wings.

Figure 5.9 illustrates the barrier sensitivity. As the barrier is further away from the initial spot price (lower for the Down-and-Out call, higher for the Up-and-Out call), the probability of the knock-out event occurring decreases. Consequently, the value of the barrier option monotonically increases, asymptotically approaching the price of an unconstrained Vanilla Call (indicated by the dashed horizontal line).

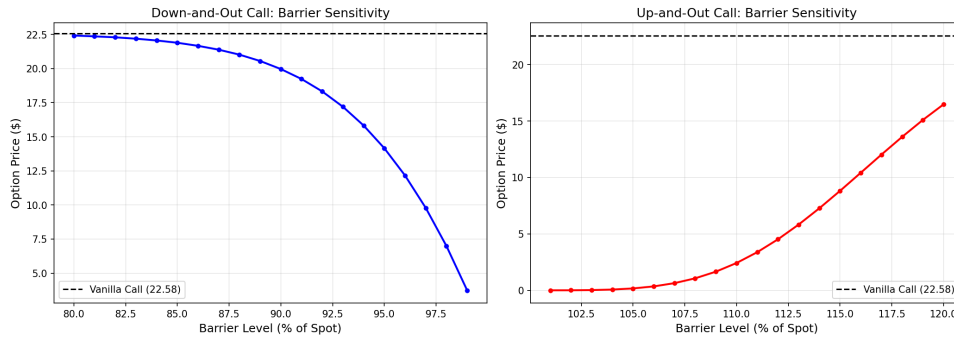


Fig. 5.9. Option price as a function of the barrier level under the calibrated local volatility model. The solid curves represent the FDM-LV prices for the Down-and-Out (left) and Up-and-Out (right) calls. The dashed line indicates the price of a standard Vanilla Call, which acts as the asymptotic upper bound for the barrier option's value.

5.4.3. Greeks smoothness and pricing speed

Figure 5.10 compares the Delta and Gamma of the barrier option computed with the PINN's autograd against those obtained using the FDM bump-and-reset method. The PINN sensitivities are visibly better and smoother. The FDM Gamma oscillates wildly due to the amplification of finite-difference noise, while the PINN Gamma remains a continuous and stable function. This validates hypothesis that automatic differentiation through a trained network naturally produces smooth Greeks.

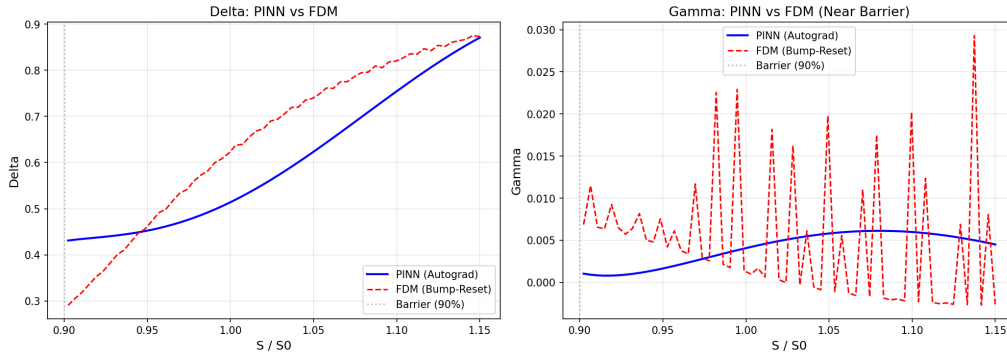


Fig. 5.10. Delta (left) and Gamma (right) for the down-and-out call as functions of the relative spot price S/S_0 . The PINN sensitivities (blue, solid) are computed via exact autograd, while the FDM sensitivities (red, dashed) are computed via numerical bump-and-reset. Near the 90% barrier limit, the FDM Gamma exhibits severe numerical instability, whereas the PINN Gamma remains perfectly smooth.

Regarding pricing speed: the PINN requires a one-time training cost that is a bit long under normal hardware, but once trained, inference is almost instantaneous: a batch of 1,000 barrier option prices is computed almost instantly. In the tests performed, a batch of 1,000 distinct barrier option configurations was computed in just 19.7 milliseconds ($19.7 \mu\text{s}$ per option) via a single batched forward pass, with no PDE solve or simulation required. The FDM engine, by contrast, has to solve a Crank–Nicolson grid for each new pricing request: at the 200×200 resolution used for this benchmark it requires 352 ms per option, so pricing the same batch would take approximately 352 seconds. This FDM figure is extrapolated from ten individually priced options rather than measured over the full batch of 1,000. This represents an execution speedup of over 17,800x, confirming the core advantage of the PINN architecture: a high computational cost for training that can be worth it because it prices rapidly over repeated, high-frequency queries (Section 4.5.2).

6. CONCLUSIONS AND FUTURE WORK

This chapter synthesises the contributions of the thesis, maps the experimental outcomes to the stated objectives and hypotheses, and identifies the most promising directions for future research.

6.1. Summary of contributions

The four-phase PINN-based pipeline has been validated for local volatility calibration and barrier option pricing.

O1 – Forward pricing validation (Phase 1). A single-output PINN was trained to solve the Black–Scholes PDE under constant volatility. Evaluated on a 200×200 grid against the closed-form solution, the network achieved $\text{RMSE} = 0.0023$ and $\text{MAE} = 0.0014$. The only relevant error is localised at the payoff kink $S = K$ at expiry, and can be explained because it cannot be reproduced using a smooth approximator. Therefore, the results confirm that the PINN architecture is reliable and, although it makes no practical sense to use it as an alternative to a direct closed-form formula, it shows that the chosen framework behaves correctly and can be applied to more complex problems where no analytical solution exists.

O2 – Inverse calibration on synthetic data (Phase 2). The architecture was extended to a dual-output network that simultaneously approximates the call price surface and the local volatility function. Trained on 250 synthetic option prices generated from a known parametric surface, the network successfully recovered the ground-truth shape: roughly 87.5% of grid cells fell within 5% relative error of the true surface and 99.5% within 10%. The five loss modifications introduced in this phase (Tikhonov anisotropic smoothness, C^2 -continuous arbitrage constraints, balanced loss weights and volatility pre-training) proved to be extremely useful.

O3 – Real-data calibration on SPY (Phase 3). The Phase 2 methodology was applied to 1,956 SPY OTM options on 20 September 2021. The price network achieved $R^2 = 0.9991$ across the moneyness range. The recovered surface is smooth, has a reasonable range of $\hat{\sigma} \in [0.08, 0.36]$, and shows zero negative-variance and zero NaNs across the evaluation grid. In contrast, raw FDM Dupire inversion produces negative local variance at 8.5% of cells (with a maximum of $\hat{\sigma} = 8.36$), and SVI-smoothed Dupire leaves 65.7% of the grid undefined due to calendar-arbitrage violations. The PINN is the only method that achieves a fully valid surface.

O4 – Barrier option pricing under the calibrated surface (Phase 4). Three independent pricing engines (Crank–Nicolson FDM, Monte Carlo with BGK correction, and a barrier-specific PINN) were cross-validated on six barrier configurations. FDM-LV and

MC-LV reached the same results for all six configurations, confirming that the calibrated surface can be used with both engines. The barrier PINN agrees with both benchmarks to within 2.2% on the three down-and-out configurations, but does not converge reliably on the three up-and-out configurations, where the final training loss is an order of magnitude higher and repeated runs of the same code produce materially different prices; as a pricing engine it is therefore validated only in the down-and-out regime. The LV surface produced huge differences relative to flat Black–Scholes (which incorrectly assumes constant volatility): down-and-out calls were priced 4–6% lower (higher knock-out probability near the put skew) and up-and-out calls 59–109% higher (lower knock-out probability in the call wing), with the magnitude growing as the barrier moves deeper out of the money.

Taken together, the four objectives support the two research hypotheses:

H1 confirmed. The PINN-calibrated surface produces barrier option prices that agree with those of FDM and MC.

H2 confirmed. The barrier PINN’s Delta and Gamma, computed via exact autograd, are smooth and continuous functions of the spot, whereas the FDM Gamma obtained via bump-and-reset oscillates severely near the barrier. Additionally, a batch of 1,000 barrier configurations is priced in 19.7 ms via a single forward pass, compared to approximately 352 s for the FDM engine – a speedup of over 17,800×

6.2. Overall Assessment

This thesis demonstrates that Physics-Informed Neural Networks are a practically viable tool for the end-to-end problem of local volatility calibration and exotic option pricing. By embedding the Dupire PDE directly into the training objective, the network jointly satisfies the pricing equation, the boundary conditions, and the market data.

The comparison against raw FDM Dupire and SVI-smoothed Dupire on real SPY data makes clear that the PINN is the only of the tested approaches that can produce 100% valid, arbitrage-free, and physically realistic results.

On the downstream pricing side, the agreement between FDM-LV and MC-LV validates the calibrated surface as a useful ‘resource’ for exotic pricing. The barrier PINN then shows that the same architecture that produces smooth calibration can also produce smooth sensitivities.

The field of PINNs applied to quantitative finance is still young, but the results presented here suggest that it is one of the most promising areas of research, with potential to completely reshape processes in the industry.

6.3. Future work

Natural extensions and next steps to this work are:

Convergence of the barrier PINN in the up-and-out regime. The Phase 4 results identify a concrete failure mode of the present formulation. The three up-and-out networks terminate with a training loss an order of magnitude higher than their down-and-out counterparts, show a non-monotonic loss over their final epochs, and are not reproducible across independent runs. The effect is most severe where the evaluation point lies closest to the barrier, precisely where the hard ansatz $g(m) = 1 - e^{-\alpha(m_B - m)}$ that enforces the knock-out condition is steepest and any residual network error is amplified multiplicatively. Adaptive loss weighting, importance sampling of collocation points near the barrier, or a softer enforcement of the knock-out condition are the natural candidates for restoring convergence in this regime.

Parametric barrier PINN. The current Phase 4 PINN is trained for a fixed barrier level, so a new training run is required for each value of B . A future extension would include the barrier level B (or the log-barrier $\ln(B/S_0)$) as an additional input dimension, producing a network that prices any barrier configuration within the trained domain in a single forward pass.

American and early-exercise options. Although the SPY options used are technically American-style, we focus on out-of-the-money contracts to eliminate the early-exercise characteristic. Handling early exercise requires solving a free-boundary (optimal stopping) problem, where the early-exercise boundary is itself an unknown.

Stochastic-local volatility (SLV) models. Using the Dupire local volatility surface for pricing options in the future assumes that today's market information about volatility will remain valid, but this is not a realistic assumption and it produces poor forward-smile dynamics. A stochastic-local volatility hybrid (like Heston) reproduces both today's surface and the vol-of-vol regime, at the cost of adding one or more unobservable stochastic factor. Extending the PINN inverse problem to recover an SLV mixing function is an open problem.

Multi-asset and basket options. The log-moneyness coordinate system can scale naturally to multi-asset settings by adding one moneyness dimension per underlying. For a two-asset basket, the PDE is two-dimensional in (m_1, m_2, τ) . This is relevant for index options, where the local correlation surface is an additional calibration target.

Loss balancing and training robustness. The five loss modifications introduced in Phase 2 involve hand-tuned hyperparameters $(\lambda_{\text{data}}, \lambda_{\text{pde}}, w_m, w_\tau, \epsilon_{\text{vega}})$. Adaptive weighting schemes, such as those proposed by Wang et al. [26], could automate this tuning and improve convergence robustness across market regimes without manual adjustment.

Extended PINNs (X-PINNs) and domain decomposition. The current network must resolve the full domain $[m_{\min}, m_{\max}] \times [0, \tau_{\max}]$ with a single pair of networks. For

very large domains, or when the solution has rapid variation in certain subregions (e.g., near short maturities), domain decomposition methods such as X-PINNs could improve training efficiency by assigning dedicated sub-networks to different parts of the domain. This would introduce the challenge of ensuring continuity between sub-domains.

Production deployment and online recalibration. The current training pipeline runs offline and produces a static calibration for a given market snapshot. In a production environment, the volatility surface must be recalibrated continuously as new quotes arrive. Starting from a previously trained network (transfer learning) can be a reasonable approach to reduce the cost of recalibration.

7. REGULATORY FRAMEWORK AND SOCIO-ECONOMIC ENVIRONMENT

The methodology proposed in this thesis has a clear target for deployment: the derivatives desks at banks and investment firms. These institutions operate within a strict regulatory environment. This chapter describes the main legal and technical constraints applicable to a production implementation of the PINN-based calibration pipeline, and analyses the economic context and implications of the research.

7.1. Regulatory Framework

7.1.1. Financial regulation and model governance

The most directly applicable regulatory framework for a quantitative pricing model is **model risk governance**. In the United States, the Federal Reserve’s SR 11-7 supervisory letter [30] establishes that banks must develop, validate, and continuously monitor all models used for material business decisions. In Europe, the European Central Bank’s Guide on Internal Models imposes analogous requirements for supervised institutions. Under both frameworks, a production volatility-surface model derived from ML techniques would be required to undergo independent model validation by a team separate from its developers.

MiFID II (Directive 2014/65/EU [31]) requires investment firms to apply sound pricing models and to demonstrate that prices derived algorithmically are fair and consistent with observable market data. The strong numerical results, such as the $R^2 = 0.9991$ price fit in Phase 3 and the absence of static-arbitrage violations in the recovered surface (Section 5.3.3), are compliant with MiFID II principles; however, this thesis presents only a single market snapshot, which is insufficient for a full validation programme. Ongoing backtesting would be required in a live environment.

The **Fundamental Review of the Trading Book** (FRTB, Basel IV [32]) governs regulatory capital for derivatives books. Under the Internal Model Approach (IMA), a local volatility surface recovered by the PINN would serve as an input risk factor for capital calculations. FRTB imposes profit-and-loss attribution tests and backtesting requirements that any such surface must satisfy. The 100% validity rate (zero negative-variance cells, zero calendar-arbitrage violations) achieved in Phase 3 is a minimum necessary condition for FRTB eligibility.

7.1.2. EU Artificial Intelligence Act

Regulation (EU) 2024/1689 [33] (the EU AI Act), in force from 2024, establishes a risk-based classification for AI systems. Annex III lists as high-risk any AI system used to assess creditworthiness or to make decisions that affect access to financial products. Derivatives pricing models used in production fall within this scope when their outputs propagate into risk-weighted asset calculations or automated hedging decisions.

If classified as high-risk, the system would be subject to requirements including technical documentation (Article 11), human oversight mechanisms (Article 14), and a conformity assessment procedure (Article 43). The design of the PINN-based pipeline has properties that are aligned with these requirements: the output is a financially interpretable local volatility surface whose values have a well-defined meaning at each point, and the Dupire PDE residual is an explicit constraint that can be verified independently of the network weights. This interpretability makes the approach relatively transparent compared to purely black-box deep calibration architectures.

7.1.3. Technical standards and open-source licensing

All numerical computations follow **IEEE 754** double-precision floating-point arithmetic, the international standard for numerical computation in financial applications. The software stack is entirely open-source:

- PyTorch is released under the BSD 3-Clause licence.
- NumPy, SciPy, and Matplotlib are released under the BSD and PSF licences.

These licences allow commercial use, modification, and redistribution without restriction, subject only to attribution requirements. There are no copyleft obligations that would constrain the licensing of derivative works.

7.1.4. Intellectual property

This work is an academic research contribution. The methodology builds on public-domain results (Dupire’s local volatility equation [1] and the PINN framework of Raissi et al. [4]) and does not constitute a claim of novel patentable subject matter. The complete source code is available in the public repository referenced in Appendix C under an open-source licence.

7.2. Socio-economic Environment

7.2.1. Project budget

Table 7.1 details the human and material resources consumed in producing this thesis. The tutor’s contribution includes motivation, advising, supervision, and revision of the work. All software dependencies are open-source and the SPY options dataset was obtained from freely available public market data sources.

The hardware and electricity costs are negligible. The laptop amortization follows this logic: The laptop was purchased for €530. The amortisation cost was estimated using straight-line depreciation over a useful life of 5 years (60 months), resulting in a monthly cost of €8.83. Since the project required approximately 6 months of work, an equipment cost of €53.00 was allocated to the project.

$$\text{Monthly amortization} = \frac{530}{12 \times 5} = 8.83 \text{ €/month}$$

$$\text{Project allocation: } 8.83 \times 6 = 53.00 \text{ €}$$

Even though the research process is relatively modest in budget, implementing this methodology in a real production scenario would require a substantially higher cost in infrastructure to operate at real-time speeds.

Concept	Unit cost	Units	Total
<i>Human resources</i>			
Student (junior engineer)	€16/h	300 h	€4,800.00
Tutor supervision (associate professor)	€40/h	20 h	€800.00
<i>Equipment</i>			
Laptop amortisation (Intel i3, 5-year life)	€8.83/mo	6 months	€53.00
<i>Software and data</i>			
Software licences (all open-source)	€0	—	€0.00
Market data (SPY options, public dataset)	€0	—	€0.00
Total			€6,653.00

TABLE 7.1. PROJECT BUDGET

7.2.2. Socio-economic impact

Economic impact. Accurate volatility surface calibration has direct economic value in derivatives markets. The calibration failures observed in the classical benchmarks (8.5% negative-variance cells for raw FDM Dupire and 65.7% undefined cells for SVI-smoothed Dupire) propagate into mispriced barrier options, incorrect delta hedges, and miscalculated regulatory capital. In a derivatives desk managing tens of billions in open posi-

tions, a systematic one-percent calibration error translates into significant hedging losses. Furthermore, if the surface contains arbitrage violations and prices breach no-arbitrage bounds, it poses a severe compliance risk under MiFID II and FRTB.

The open-source release of the calibration pipeline makes a complete, working implementation accessible to smaller institutions and academic researchers who lack access to commercial volatility surface tools, reducing barriers to entry in quantitative finance research and enabling reproducibility of the results.

Social and ethical impact. The deployment of ML models in financial decision-making tends to raise concerns about opacity and accountability. The PINN approach mitigates some of these concerns relative to purely black-box deep calibration: the output is a financially interpretable function, and the Dupire PDE residual is an explicit, verifiable constraint. The model does not make autonomous trading decisions; it simply provides a calibrated surface that feeds into human-supervised pricing and risk workflows. Nonetheless, the model risk governance requirements discussed in Section 7.1.1 apply, and production deployment without independent validation would not be appropriate under either SR 11-7 or EU AI Act requirements.

Environmental impact. All training was performed on a CPU-only laptop without cloud computing. The total training time across all the attempts for the four phases is approximately 30 hours, at a laptop power draw of roughly 15 W. Including all development, testing, and writing activities, a broad approximation of total energy consumption is well under 10 kWh, which is orders of magnitude below the footprint of large-scale GPU-based deep learning pipelines. The approach is lightweight by design, as the network architectures are compact.

BIBLIOGRAPHY

- [1] B. Dupire, “Pricing with a smile,” *Risk*, vol. 7, no. 1, pp. 18–20, 1994.
- [2] S. L. Heston, “A closed-form solution for options with stochastic volatility with applications to bond and currency options,” *The Review of Financial Studies*, vol. 6, no. 2, pp. 327–343, 1993.
- [3] P. S. Hagan, D. Kumar, A. S. Lesniewski, and D. E. Woodward, “Managing smile risk,” *Wilmott Magazine*, pp. 84–108, 2002.
- [4] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.
- [5] M. Rubinstein and E. Reiner, “Breaking down the barriers,” *Risk*, vol. 4, no. 8, pp. 28–35, 1991.
- [6] F. Black and M. Scholes, “The pricing of options and corporate liabilities,” *Journal of Political Economy*, vol. 81, no. 3, pp. 637–654, 1973.
- [7] R. C. Merton, “Theory of rational option pricing,” *Bell Journal of Economics and Management Science*, vol. 4, no. 1, pp. 141–183, 1973.
- [8] M. Rubinstein, “Implied binomial trees,” *The Journal of Finance*, vol. 49, no. 3, pp. 771–818, 1994.
- [9] E. Derman and I. Kani, “Riding on a smile,” *Risk*, vol. 7, no. 2, pp. 32–39, 1994.
- [10] J. Hadamard, *Lectures on Cauchy’s Problem in Linear Partial Differential Equations*. New Haven: Yale University Press, 1923.
- [11] M. Broadie, P. Glasserman, and S. G. Kou, “A continuity correction for discrete barrier options,” *Mathematical Finance*, vol. 7, no. 4, pp. 325–349, 1997.
- [12] G. Cybenko, “Approximation by superpositions of a sigmoidal function,” *Mathematics of Control, Signals and Systems*, vol. 2, no. 4, pp. 303–314, 1989.
- [13] K. Hornik, “Approximation capabilities of multilayer feedforward networks,” *Neural Networks*, vol. 4, no. 2, pp. 251–257, 1991.
- [14] D. P. Kingma and J. Ba, *Adam: A method for stochastic optimization*, arXiv preprint arXiv:1412.6980, 2014.
- [15] I. E. Lagaris, A. Likas, and D. I. Fotiadis, “Artificial neural networks for solving ordinary and partial differential equations,” *IEEE Transactions on Neural Networks*, vol. 9, no. 5, pp. 987–1000, 1998.
- [16] J. Gatheral, *The Volatility Surface: A Practitioner’s Guide*. John Wiley & Sons, 2006.

- [17] J. Gatheral, “A parsimonious arbitrage-free implied volatility parameterization with application to the valuation of volatility derivatives,” in *Presentation at Global Derivatives & Risk Management*, 2004.
- [18] J. Gatheral and A. Jacquier, “Arbitrage-free svi volatility surfaces,” *Quantitative Finance*, vol. 14, no. 1, pp. 59–71, 2014.
- [19] J. Gatheral, T. Jaisson, and M. Rosenbaum, “Volatility is rough,” *Quantitative Finance*, vol. 18, no. 6, pp. 933–949, 2018.
- [20] S. Crépey, “Calibration of the local volatility in a generalized black-scholes model using tikhonov regularization,” *SIAM Journal on Mathematical Analysis*, vol. 34, no. 5, pp. 1183–1206, 2003.
- [21] H. Egger and H. W. Engl, “Tikhonov regularization applied to the inverse problem of option pricing: Convergence analysis and rates,” *Inverse Problems*, vol. 21, no. 3, pp. 1027–1045, 2005.
- [22] J. M. Hutchinson, A. W. Lo, and T. Poggio, “A nonparametric approach to pricing and hedging derivative securities via learning networks,” *The Journal of Finance*, vol. 49, no. 3, pp. 851–889, 1994.
- [23] H. Buehler, L. Gonon, J. Teichmann, and B. Wood, “Deep hedging,” *Quantitative Finance*, vol. 19, no. 8, pp. 1271–1291, 2019.
- [24] J. Han, A. Jentzen, and W. E, “Solving high-dimensional partial differential equations using deep learning,” *Proceedings of the National Academy of Sciences*, vol. 115, no. 34, pp. 8505–8510, 2018.
- [25] J. Sirignano and K. Spiliopoulos, “Dgm: A deep learning algorithm for solving partial differential equations,” *Journal of Computational Physics*, vol. 375, pp. 1339–1364, 2018.
- [26] S. Wang, Y. Teng, and P. Perdikaris, “Understanding and mitigating gradient pathologies in physics-informed neural networks,” *SIAM Journal on Scientific Computing*, vol. 43, no. 5, A3055–A3081, 2021.
- [27] L. Andersen and R. Brotherton-Ratcliffe, “The equity option volatility smile: An implicit finite-difference approach,” *The Journal of Computational Finance*, vol. 1, no. 2, pp. 5–38, 1997.
- [28] D. Tavella and C. Randall, *Pricing Financial Instruments: The Finite Difference Method*. John Wiley & Sons, 2000.
- [29] P. Glasserman, *Monte Carlo Methods in Financial Engineering*. Springer, 2004.
- [30] Board of Governors of the Federal Reserve System, *SR 11-7: Guidance on Model Risk Management*, Supervisory Letter SR 11-7, 2011. [Online]. Available at: <https://www.federalreserve.gov/supervisionreg/srletters/sr1107.htm>

- [31] European Parliament and Council of the European Union, *Directive 2014/65/EU on markets in financial instruments (MiFID II)*, Official Journal of the European Union, L 173/349, 2014.
- [32] Basel Committee on Banking Supervision, *Minimum capital requirements for market risk*, BIS publication, 2019. [Online]. Available at: <https://www.bis.org/bcbs/publ/d457.htm>
- [33] European Parliament and Council of the European Union, *Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act)*, Official Journal of the European Union, L 2024/1689, 2024.
- [34] J. C. Hull, *Options, Futures, and Other Derivatives*, 10th ed. Pearson, 2018.
- [35] E. G. Haug, *The Complete Guide to Option Pricing Formulas*, 2nd ed. McGraw-Hill, 2007.
- [36] Y.-K. Kwok, *Mathematical Models of Financial Derivatives*, 2nd ed. Springer, 2008.

A. MATHEMATICAL DERIVATIONS

This annex presents a summarized version of the standard derivations of the main mathematical concepts used. Classical derivations of the Black–Scholes PDE can be found in [6], [7], [34]. Barrier option pricing formulas follow the work in [5], [35], [36]. The continuity correction for discretely monitored barrier options is based on [11].

A.0.1. Derivation of the Black-Scholes PDE

We assume the underlying asset price S_t follows a Geometric Brownian Motion (GBM):

$$dS_t = \mu S_t dt + \sigma S_t dW_t \quad (\text{A.1})$$

where μ is the drift, σ is the constant volatility, and W_t is the Wiener process.

Let $V(S, t)$ be the price of an European option. If we apply Itô's Lemma to $V(S, t)$, we get that the change in option prices is given by:

$$dV = \left(\frac{\partial V}{\partial t} + \mu S \frac{\partial V}{\partial S} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \right) dt + \sigma S \frac{\partial V}{\partial S} dW_t \quad (\text{A.2})$$

Now, assume Π represents a portfolio consisting of one short position in the option and Δ shares of the underlying asset:

$$\Pi = -V + \Delta S \quad (\text{A.3})$$

The change in this portfolio value over an infinitesimal time step dt is:

$$d\Pi = -dV + \Delta dS \quad (\text{A.4})$$

Now we can substitute dS_t and dV in terms of Π , and solving for that term we get:

$$d\Pi = - \left(\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \right) dt + \left(\Delta - \frac{\partial V}{\partial S} \right) (\mu S dt + \sigma S dW_t) \quad (\text{A.5})$$

By definition, Δ defines the sensitivity of the option price w.r.t. the asset price, $\Delta = \frac{\partial V}{\partial S}$. Therefore, all the second part can be eliminated:

$$d\Pi = - \left(\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \right) dt \quad (\text{A.6})$$

Under the no-arbitrage principle, a risk-free portfolio (and this is now a risk free portfolio, as we have eliminated dW_t) must earn the risk-free interest rate r , such that $d\Pi = r\Pi dt$. Substituting the two equations we have for $d\Pi$, A.4 and A.6, we get:

$$- \left(\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \right) dt = r \left(-V + \frac{\partial V}{\partial S} S \right) dt \quad (\text{A.7})$$

Dividing by dt and rearranging the terms, we arrive at the Black-Scholes PDE:

$$\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0 \quad (\text{A.8})$$

A.0.2. Analytical Closed-Form Formulas for Barrier Options

Pricing path-dependent barrier options under constant volatility relies on the reflection principle of Brownian motion⁶. We assume the stock price follows geometric Brownian motion, $dS_t = rS_t dt + \sigma S_t dW_t$, which can be expressed using logarithms as $\ln S_t = \ln S_0 + \left(r - \frac{1}{2}\sigma^2\right)t + \sigma W_t$.

The reflection principle allows the contribution of all paths that hit the barrier to be expressed through a “mirrored” process. Rearranging the general reflection formula⁷ using logs and the terminology of our case (B as the mirror point), we get the effective starting price: $\ln(B^2/S_0) = 2 \ln B - \ln S_0$.

We define the auxiliary quantities:

$$\lambda = \frac{r + \frac{1}{2}\sigma^2}{\sigma^2}, \quad \begin{aligned} x_1 &= \frac{\ln(S_0/B)}{\sigma \sqrt{T}} + \lambda \sigma \sqrt{T}, & x_2 &= x_1 - \sigma \sqrt{T}, \\ y_1 &= \frac{\ln(B/S_0)}{\sigma \sqrt{T}} + \lambda \sigma \sqrt{T}, & y_2 &= y_1 - \sigma \sqrt{T}. \end{aligned}$$

Let $C_{BS}(S, K, T)$ and $P_{BS}(S, K, T)$ denote the standard Black-Scholes Call and Put prices. For a Down-and-Out Call (DOC) with $S_0 > B$ and $K > B$, the option value can be conceptually decomposed into the standard call minus the contribution of the knocked-out paths:

$$C_{DOC} = C_{BS}(S_0, K, T) - (\text{contribution of knocked-out paths}). \quad (\text{A.9})$$

Conceptually, we want to compute the contribution of these knocked-out paths: all trajectories that start at S_0 , hit the barrier at some moment, and eventually finish above K (if they do not finish above K , their payoff is zero). For every path starting at S_0 and touching the barrier, there is exactly one mirrored path starting at B^2/S_0 that touches the barrier at the same time, since they are at an equal logarithmic distance. From that moment of intersection, the probability of both paths reaching K is identical.

Therefore, the probability of a path starting at S_0 , touching the barrier, and finishing above K is equal to the probability of a standard call option path starting at B^2/S_0 and finishing above K (as all these paths inevitably must cross the barrier). This perfect symmetry holds in a world without drift. Because we model the stock with a drift r , we must multiply this mirrored option value by an adjustment factor to account for the upward bias. The final evaluated expression is:

$$C_{DOC} = C_{BS}(S_0, K, T) - C_{BS}\left(\frac{B^2}{S_0}, K, T\right) \left(\frac{S_0}{B}\right)^{1-\frac{2r}{\sigma^2}}. \quad (\text{A.10})$$

⁶The reflection principle states that for a Brownian motion path that touches a threshold at a moment t , there is an equal probability of the path ending above or below the threshold. Consequently, the probability of crossing a barrier can be related to the probability of ending beyond it.

⁷The general reflection formula follows from symmetry around a point m . If x is reflected across m , the reflected point x_{ref} must satisfy $m - x = x_{\text{ref}} - m$. Rearranging gives $x_{\text{ref}} = 2m - x$.

Similarly, for an Up-and-Out Call (UOC) with $S_0 < B$ and $K < B$, additional correction terms appear because the payoff must be truncated in the region near the barrier. The final expression becomes:

$$C_{UOC} = C_{BS}(S_0, K, T) - C_{BS}(S_0, B, T) - (B - K)e^{-rT} \mathcal{N}(x_2) - \left(\frac{S_0}{B}\right)^{1-\frac{2r}{\sigma^2}} \left[C_{BS}\left(\frac{B^2}{S_0}, K, T\right) - C_{BS}\left(\frac{B^2}{S_0}, B, T\right) - (B - K)e^{-rT} \mathcal{N}(y_2) \right], \quad (\text{A.11})$$

where $\mathcal{N}(\cdot)$ is the cumulative distribution function of the standard normal distribution.

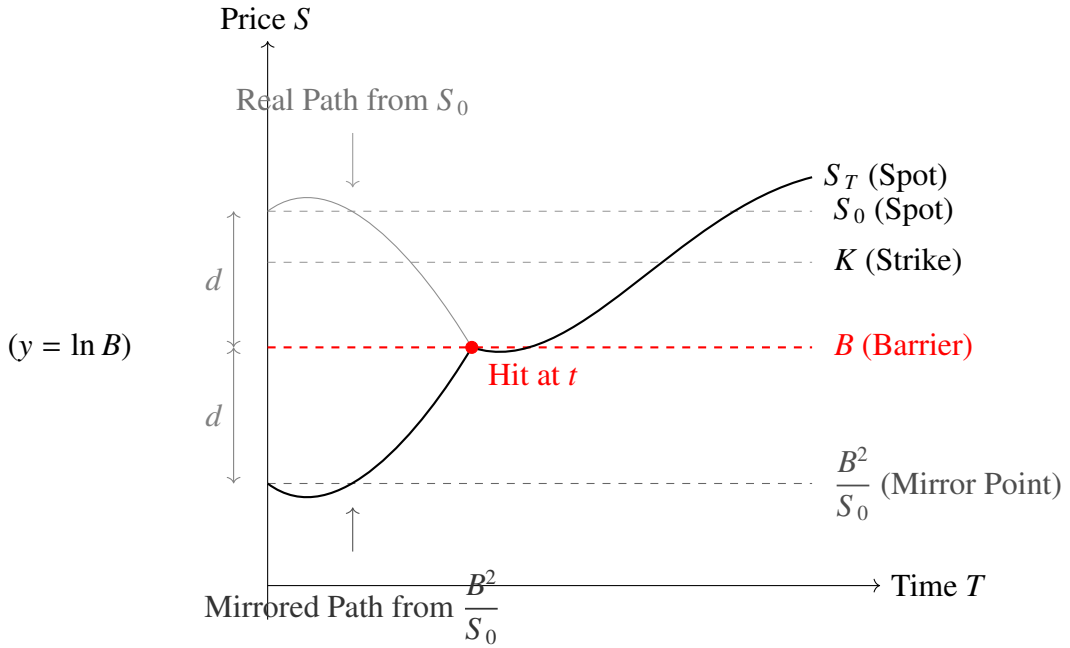


Fig. A.1. Visualization of the Reflection Principle for a DOC. A path starting at S_0 that crosses the barrier B has a symmetric path starting at B^2/S_0 (black) that also crosses B . The two distances d are equal: $d = \ln(S_0/B) = \ln(B/B^2/S_0)$.

A.0.3. The Broadie-Glasserman-Kou (BGK) Continuity Correction

Analytical formulas are continuous, so if an asset crosses the barrier it is because at some point the asset has exactly the value of the barrier. However, in practice and in MC simulations, the asset path is observed at discrete time intervals Δt . This allows the asset price to 'jump' the barrier between observation points without triggering a knock-out (having exactly the value of the barrier).

Broadie, Glasserman, and Kou [11] demonstrated that a discretely monitored barrier option can be priced using continuous formulas by replacing the barrier B by an effective

barrier B_{eff} . For a monitoring frequency of Δt , the effective barrier is:

$$B_{eff} = B \cdot \exp\left(\pm\beta\sigma\sqrt{\Delta t}\right) \quad (\text{A.12})$$

where the constant β is derived from the Riemann zeta function:

$$\beta = -\frac{\zeta\left(\frac{1}{2}\right)}{\sqrt{2\pi}} \approx 0.5826 \quad (\text{A.13})$$

The sign is positive (+) for an Up-and-Out barrier (shifting the barrier upward) and negative (−) for a Down-and-Out barrier (shifting the barrier downward). Conceptually, what we are doing is mathematically tightening the knock-out region to compensate for the discrete observation intervals.

B. IMPLEMENTATION DETAILS

This appendix provides the full hyperparameter tables, network architectures, and training configurations for all four experimental phases. These details are omitted from Chapter 4 to preserve readability.

B.1 Computational Environment

Component	Specification
CPU	Intel Core i3-1115G4 @ 3.00 GHz (dual-core, 4 threads)
RAM	8 GB
GPU	None (Intel UHD Graphics integrated; all training on CPU)
OS	Windows 11 Home
Python	3.10
PyTorch	≥ 2.0
NumPy	≥ 1.24
SciPy	≥ 1.10
Matplotlib	≥ 3.7
Random seed	42 (all phases)

TABLE B.1. HARDWARE AND SOFTWARE ENVIRONMENT

B.2 Network Architectures

Phase	Network	Input dim.	Layers	Width	Output / activation
1	Price net	2 ($S_{\text{norm}}, t_{\text{norm}}$)	4	64	$\hat{V}$ – linear
2–3	Price net	2 ($m_{\text{norm}}, \tau_{\text{norm}}$)	4	64	$\hat{v}$ – linear
2–3	Vol net	2 ($m_{\text{norm}}, \tau_{\text{norm}}$)	3	64	$\hat{\sigma} = \text{softplus}(\cdot) + 0.01$
4	Barrier PINN	2 ($m_{\text{norm}}, \tau_{\text{norm}}$)	4	64	$\hat{v} \cdot g(m)$ – hard ansatz

All hidden layers use tanh activation. $g(m) = 1 - e^{-\alpha(m-m_B)}$, $\alpha = 10$.

TABLE B.2. NEURAL NETWORK ARCHITECTURES BY PHASE

B.3 Phase 1 Hyperparameters

Parameter	Value	Description
<i>Option parameters</i>		
K	50.0	Strike price
r	0.05	Risk-free rate
q	0.015	Dividend yield
σ	0.25	Constant volatility
T	0.5	Time to expiry (years)
$[S_{\min}, S_{\max}]$	$[0, 100]$	Spot price domain
<i>Collocation points</i>		
N_{pde}	10,000	Interior (PDE residual)
N_{ic}	500	Terminal condition ($t = T$)
N_{bc}	200	Boundary (100 lower + 100 upper)
<i>Loss weights</i>		
λ_{pde}	1.0	PDE residual
λ_{ic}	10.0	Terminal condition
λ_{bc}	1.0	Boundary conditions
<i>Optimisers</i>		
Adam epochs	8,000	–
Adam lr	10^{-3}	Cosine annealing to 10^{-5}
L-BFGS iters	2,000	Strong Wolfe line search
L-BFGS lr	1.0	–
L-BFGS history	50	–
Resample every	2,000 epochs	Adam collocation resampling
<i>Training time</i>		
Adam	≈ 32 min	–
L-BFGS	≈ 11.5 min	–
Total	≈ 45 min	–

TABLE B.3. PHASE 1 – BLACK–SCHOLES PARAMETERS AND TRAINING CONFIGURATION

B.4 Phase 2 Hyperparameters

Parameter	Value	Description
<i>Training data</i>		
Synthetic prices	250	Generated by FDM from $\sigma_{\text{true}}(m, \tau)$
Domain $[m_{\min}, m_{\max}]$	$[-0.30, 0.30]$	Log-moneyness range
Domain $[0, \tau_{\max}]$	$[0, 1.0]$	Time-to-expiry range
<i>Collocation points</i>		
N_{pde}	10,000	Interior (frozen for L-BFGS)
N_{ic}	300	Terminal condition ($\tau = 0$)
<i>Loss weights</i>		
λ_{data}	10.0	Data fit (MSE on normalised prices)
λ_{pde}	5.0	Dupire PDE residual
λ_{smooth}	0.001	Tikhonov smoothness on $\hat{\sigma}$
λ_{ic}	3.0	Terminal condition
λ_{arb}	5.0	Arbitrage penalty (calendar + butterfly)
w_m	0.01	Smoothness weight – moneyness direction
w_τ	1.0	Smoothness weight – time direction
<i>Vol network pre-training</i>		
Epochs	500	Flat surface target $\hat{\sigma} = 0.30$
<i>Optimisers</i>		
Adam epochs	15,000	–
Adam lr	10^{-3}	Cosine annealing to 10^{-5}
L-BFGS iters	5,000	Strong Wolfe, frozen collocation
L-BFGS lr	0.1	–
L-BFGS history	50	–
<i>Training time</i>		
Total	≈ 106 min (6,361 s)	–

TABLE B.4. PHASE 2 – SYNTHETIC INVERSE CALIBRATION CONFIGURATION

B.5 Phase 3 Hyperparameters

Parameter	Value	Description
<i>Market data</i>		
Dataset	spy_otm_2021-09-20.csv	SPY options, 20 September 2021
Quotes	1,956	OTM calls and puts
Moneyness range	$[-0.25, 0.22]$	$m = \ln(S/K)$
Maturity range	$[7, 365]$ DTE	$\tau \in [0, 1.0]$ yr
<i>Collocation points</i>		
N_{pde}	10,000	Interior (frozen for L-BFGS)
N_{ic}	300	Terminal condition ($\tau = 0$)
<i>Loss weights</i>		
λ_{data}	10.0	Vega-weighted data fit
λ_{pde}	5.0	Dupire PDE residual
λ_{smooth}	0.001	Tikhonov smoothness on $\hat{\sigma}$
λ_{ic}	3.0	Terminal condition
λ_{arb}	1.0	Arbitrage penalty (calendar + butterfly)
w_m	0.01	Smoothness weight – moneyness direction
w_τ	1.0	Smoothness weight – time direction
$\varepsilon_{\text{vega}}$	10^{-3}	Vega floor for inverse-weighting
<i>Vol network pre-training</i>		
Epochs	500	Flat surface target $\hat{\sigma} = 0.20$
<i>Optimisers</i>		
Adam epochs	15,000	–
Adam lr	10^{-3}	Cosine annealing to 10^{-5}
L-BFGS iters	5,000	Strong Wolfe, frozen collocation
L-BFGS lr	0.1	–
L-BFGS history	50	–
<i>Training time</i>		
Pre-training	<1 min	Vol net to flat surface
Adam	≈ 120 min	–
L-BFGS	≈ 41 min	–
Total	≈ 165 min	–

TABLE B.5. PHASE 3 – SPY MARKET CALIBRATION
CONFIGURATION

B.6 Phase 4 Hyperparameters

Parameter	Value	Description
<i>Barrier PINN training (per barrier level)</i>		
Adam epochs	5,000	–
Adam lr	10^{-3}	–
N_{pde}	2,000	PDE collocation points
N_{ic}	500	Terminal condition points
Ansatz steepness α	10	Hard boundary: $g(m) = 1 - e^{-10(m-m_B)}$
Typical training time	43–85 min	Measured per barrier level (5,000 Adam epochs, CPU only)
Greeks demo time	≈91 min	5,000 epochs full precision (5,444 s)
<i>FDM benchmark</i>		
Grid size	600×600	Strike $\times$ time steps
Scheme	Crank–Nicolson	–
<i>Monte Carlo benchmark</i>		
Paths	100,000	Antithetic variates enabled
Time steps	500	Per path
Correction	BGK	Broadie–Glasserman–Kou continuity correction

TABLE B.6. PHASE 4 – BARRIER OPTION PINN AND BENCHMARK ENGINES

B.7 Training Time Summary

Phase	Time	Notes
Phase 1 – BS PINN	≈45 min	8,000 Adam + 2,000 L-BFGS
Phase 2 – Synthetic inverse	≈106 min	15,000 Adam + 5,000 L-BFGS
Phase 3 – SPY calibration	≈165 min	Pre-training + 15,000 Adam + 5,000 L-BFGS
Phase 4 – Barrier (standard)	43–85 min/level	5,000 Adam per barrier level
Phase 4 – Greeks demo	≈91 min	Single barrier, full-precision run

TABLE B.7. TOTAL TRAINING TIMES ON THE REFERENCE HARDWARE (INTEL CORE I3-1115G4, CPU ONLY)

C. CODE REPOSITORY STRUCTURE

All the code of the thesis is available at [GitHub](#). The repository follows a phase-based structure, where each phase has an independent folder and shared utilities are placed in a common folder. The complete structure is described below.

Top-level layout

- `phase1_direct/` — Phase 1: forward Black–Scholes pricing PINN.
- `phase2_inverse/` — Phase 2: synthetic inverse calibration PINN.
- `phase3_market/` — Phase 3: real-market SPY calibration.
- `phase4_barriers/` — Phase 4: barrier option pricing.
- `utils/` — Shared utilities used across all phases.
- `results/` — Output figures and saved model checkpoints, organised by phase.
- `requirements.txt` — Python package dependencies.

`utils/`

`black_scholes.py` Black–Scholes closed-form pricing formula, implied volatility inversion via Brent’s method, and Reiner–Rubinstein constant-vol barrier option pricing.

`normalization.py` MinMax scaler and log-moneyness coordinate transform ($m = \ln(S/K)$, $\tau = T - t$).

`plotting.py` Reusable functions for surface plots, slice comparisons, and heatmaps used in all four validation scripts.

`phase1_direct/`

`pinn_bs.py` Single-output PINN architecture: four hidden layers of width 64 with tanh activations. Defines the Black–Scholes PDE residual, initial condition loss, and boundary condition loss.

`train_phase1.py` Two-stage training loop (Adam followed by L-BFGS). Saves the trained model to `results/phase1/`.

validate_phase1.py Evaluates the trained model on a 200×200 grid, computes RMSE/MAE/MaxError against the closed-form solution, and generates the error heatmap and slice comparison figures.

phase2_inverse/

pinn_dual.py Dual-output PINN: independent price network (4×64 tanh) and volatility network (3×64 tanh with softplus floor), coupled through the Dupire PDE residual.

lv_surface.py Definition of the synthetic local volatility ground truth: $\sigma(m, \tau) = 0.2 + 0.1 e^{-m^2/0.3} + 0.05\tau$.

fdm_solver.py Crank–Nicolson finite-difference solver used to generate the 250 synthetic option prices from the ground-truth surface.

loss_balancer.py Utilities for the five loss modifications: anisotropic Tikhonov smoothness, C^2 -continuous arbitrage penalties, and vega-inverse data weighting.

train_phase2.py Phase 2 training loop with volatility network pre-training and frozen L-BFGS collocation.

phase3_market/

spy_otm_2021-09-20.csv Preprocessed SPY OTM option quotes: 1,956 rows, moneyness $\in [0.80, 1.20]$, DTE $\in [7, 365]$. This is the primary real-market dataset used in Phase 3.

preprocess.py Data loading, OTM filtering, implied volatility calculation, and vega computation from the raw option chain.

data_loader.py PyTorch Dataset wrapper that converts the preprocessed quotes into normalised (m, τ, v) training tensors.

train_phase3.py Phase 3 training loop. Extends Phase 2 with vega-weighted data loss, anisotropic smoothness weights ($w_m = 0.01$, $w_\tau = 1.0$), and 15,000 Adam epochs followed by 5,000 L-BFGS iterations.

validate_phase3.py Evaluates the calibrated surface: price fit (R^2), surface validity check, and generation of the LV surface and price-fit figures.

benchmark_dupire.py Implements the two classical benchmarks (raw FDM Dupire and SVI-smoothed Dupire) and generates the three-panel heatmap and cross-section comparison figures.

phase4_barriers/

barrier_pinn.py Barrier-specific PINN that extends the dual-output architecture with a multiplicative hard-enforcement ansatz $g(m) = 1 - e^{-\alpha(m-m_B)}$ to satisfy the knock-out boundary condition by construction.

barrier_fdm.py Crank–Nicolson FDM solver for barrier options under the calibrated local volatility surface.

barrier_mc.py Monte Carlo barrier option pricer with antithetic variates and the Broadie–Glasserman–Kou continuity correction for discrete monitoring bias.

bs_barrier_closedform.py Reiner–Rubinstein closed-form barrier option pricing under constant volatility, used for all three-engine benchmarking under constant-vol assumptions.

validate_phase4.py Four-way price comparison (FDM-LV, MC-LV, PINN-LV, BS-const), Greeks comparison, barrier sensitivity curves, and pricing speed benchmark.

results/

Each subdirectory (phase1/ through phase4/) contains the figures generated.